

Justvengers: Batched VOLE ZK Disjunctions in $\mathcal{O}(R+B+C)$ Communication

Yibin Yang^[0000–0001–6062–3531]

Georgia Institute of Technology and NTT Research, Atlanta, USA
yyang811@gatech.edu

Abstract. Recent progress on zero-knowledge proofs (ZKPs) based on vector oblivious linear evaluation (VOLE) offers a promising paradigm for scaling ZKPs over extremely large statements. In particular, VOLE-based ZK is currently the best choice in terms of end-to-end execution time. However, VOLE-based ZK incurs high communication overhead — it usually scales linearly with the circuit size.

To mitigate this, existing literature considers VOLE-based ZK over structured statements. In this work, we focus on the *batched disjunctive statement* — $\mathcal{P}$ and $\mathcal{V}$ agree on B fan-in 2 circuits $\mathcal{C}_1, \dots, \mathcal{C}_B$ over a field $\mathbb{F}$; each circuit is of size C with n_{in} inputs. $\mathcal{P}$'s goal is to demonstrate the knowledge of R witnesses $(id_j \in [B], \mathbf{w}_j \in \mathbb{F}^{n_{in}})$ for each $j \in [R]$ s.t. $\forall j \in [R], \mathcal{C}_{id_j}(\mathbf{w}_j) = 0$ where neither $\mathbf{w}_j$ nor id_j is revealed. Batched disjunctive statements are effective, e.g., in emulating the CPU execution inside ZK. Note, the naïve solution results in a circuit of size $\mathcal{O}(RBC)$.

To prove such a statement using VOLE-based ZK, the prior state-of-the-art protocol **Antman** (Weng et al., CCS'22) incurred $\mathcal{O}(BC + R)$ communication by additionally relying on AHE, whereas **Batchman** (Yang et al., CCS'23) achieved $\mathcal{O}(RC + B)$ communication using only VOLE.

In this work, we combine these two protocols non-trivially and present a novel protocol **Justvengers** — targeting the batched disjunctive statement — that incurs only $\mathcal{O}(R + B + C)$ communication and $\mathcal{O}(BC + (B + C)R \log R)$ computation for prover, using AHE and VOLE. As in **Antman**, **Justvengers** requires an AHE scheme that achieves *linear targeted malleability*, which is a non-falsifiable assumption.

1 Introduction

Zero-knowledge proofs (ZKPs) [GMR85] allow a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of the truth of a statement without revealing any information beyond its validity. ZKPs enable many applications, including private blockchains [BCG⁺14], privacy-preserving program analysis [FDNZ21, LAH⁺22], private bug bounty systems [HYDK21, YHKD22], secure machine learning [LXZ21, WYX⁺21], and many more. The last decade has seen rapid advances in the design, implementation, and practical deployment of concretely efficient ZKP systems.

VOLE-based ZK. A recent and increasingly popular line of research constructs ZKP protocols using *vector linear oblivious evaluation* (VOLE), known as

VOLE-based ZK; see, e.g., [BBMH⁺21, BBMHS22, BCC⁺24, BMRS21, DEGL⁺23, DIO21, DILO22, HY24, LHLA25, LXY24a, LXY24b, WYKW21, YSWW21]. This direction is enabled by the efficient generation of VOLE correlations — i.e., random VOLE instances — using the PCG paradigm (see, e.g., [BCGI18, BCG⁺19, SGRR19, YWL⁺20]). In VOLE-based ZK, once the cryptographic overhead of generating VOLE correlations is handled, the remaining protocol is typically simple, often relying on lightweight or even information-theoretic cryptography, and can be made extremely efficient in practice.

Consider a fan-in 2 circuit $\mathcal{C}$ over a finite field, with C denoting the total number of gates. VOLE-based ZK over $\mathcal{C}$ typically requires cost (i.e., communication and computation of each party) of a small constant factor over C in terms of (extension) field elements and operations. State-of-the-art VOLE-based ZK systems (e.g., QuickSilver [YSWW21]) can evaluate millions of multiplication gates per second on modest hardware and network settings. Because of its high scalability, VOLE-based ZK has been especially effective in applications involving large statements, including privacy-preserving machine learning [WYX⁺21], secure static analysis [LAH⁺22, LJA⁺22, LKA⁺24], private string matching [LWS⁺24], and privacy-preserving databases [LWX⁺23].

We focus on VOLE-based ZK since it offers the fastest end-to-end proof time among all known ZKP approaches (e.g., zkSNARKs, MPC-in-the-head), enabling the efficient handling of statements at unprecedented scale — such as those in the applications mentioned above. For further discussion, see Section 1.2.

Batched disjunctive statements. In this work, we are interested in designing VOLE-based ZK protocols targeting the *batched disjunctive statements*. Here, $\mathcal{P}$ and $\mathcal{V}$ agree on B fan-in 2 circuits $\mathcal{C}_1, \dots, \mathcal{C}_B$ over a field $\mathbb{F}$; each circuit is of size C with n_{in} inputs. $\mathcal{P}$'s goal is to demonstrate the knowledge of R witnesses ($id_j \in [B]$, $\mathbf{w}_j \in \mathbb{F}^{n_{in}}$) for each $j \in [R]$ s.t. $\forall j \in [R], \mathcal{C}_{id_j}(\mathbf{w}_j) = 0$ where neither $\mathbf{w}_j$ nor id_j is revealed. We refer to each of the B circuits as a *branch*, and the one for which $\mathcal{P}$ holds a valid witness as the *active branch*.

Batched disjunctions are, in fact, common in computation — e.g., a physical CPU repeatedly evaluates the same disjunction over its instruction set as it executes each step of a program. Efficiently supporting such batched disjunctions in ZK is therefore a natural and important building block toward concrete ZK proofs (or arguments) for RAM programs (see, e.g., [GHAK24, IOS23, IOS25, YHH⁺24]). Our work continues this direction.

The naïve way to proving a disjunctive statement is to evaluate all branches independently and then use a *multiplexer* to select the output of the active branch. Repeating this multiplexed structure R times yields a circuit of size $\mathcal{O}(RBC)$ — inferring $\mathcal{O}(RBC)$ communication in linear-cost VOLE-based ZK.

Given the importance of batched disjunctive statements, prior work has explored ways to improve their efficiency in VOLE-based ZK. In particular, the Batchman protocol [YHH⁺23a] achieves $\mathcal{O}(RC + B)$ communication with (sublinear) computation costs of $\mathcal{O}(BC + RC)$ for both parties, relying solely on VOLE. Also, the Antman protocol [WYY⁺22] targets improvements for SIMD circuits in VOLE-based ZK, which implicitly improves batched disjunctive state-

Protocol	Comm.	$\mathcal{P}$'s Comp.	$\mathcal{V}$'s Comp.	Needs AHE?
QuickSilver [YSWW21]	$\mathcal{O}(RBC)$	$\mathcal{O}(RBC)$	$\mathcal{O}(RBC)$	✗
Batchman [YHH ⁺ 23a]	$\mathcal{O}(RC + B)$	$\mathcal{O}(BC + RC)$	$\mathcal{O}(BC + RC)$	✗
Antman [WYY ⁺ 22]	$\mathcal{O}(BC + R)$	$\mathcal{O}(BCR \log R)$	$\mathcal{O}(BC + R \log R)$	✓
Justvengers ours	$\mathcal{O}(R + B + C)$	$\mathcal{O}(BC + (B + C)R \log R)$	$\mathcal{O}(BC + R \log R)$	✓

Table 1: The performance of our protocol **Justvengers**, compared with the prior VOLE-based ZK work in handling batched disjunctive statements. Here, R is the number of repetitions, B is the number of branches, and each branch is a circuit of size C . Communication is measured in field elements, while computation is measured in field operations. Note that the complexity of **Batchman** [YHH⁺23a] incorporates the ZK SET [YH24] optimization; see [YHH⁺23b] for details.

ments where the common circuit has size $\mathcal{O}(BC)$. Applied in this setting, **Antman** achieves $\mathcal{O}(BC + R)$ communication, with $\mathcal{P}$ performing $\mathcal{O}(BCR \log R)$ and $\mathcal{V}$ performing $\mathcal{O}(BC + R \log R)$ computation, using VOLE and — additionally — AHE. While **Batchman** outperforms **Antman** in $\mathcal{P}$'s computation, these two approaches are ultimately incomparable in terms of communication complexity.

We review more on non-batched and batched ZK disjunctions in Section 1.2.

Cost model and field setting. In this work, we present our construction using circuits defined over a sufficiently large field $\mathbb{F}$, where the field size $|\mathbb{F}|$ is super-polynomial (i.e., $|\mathbb{F}| = \lambda^{\omega(1)}$). Communication is measured in the number of field elements, and computation is measured in the number of field operations.

However, to the best of our knowledge, all VOLE-based ZK protocols — including ours — can also be instantiated over small fields, such as the Boolean field $\mathbb{F}_2$, by embedding them into a sufficiently large extension field.¹ For the sake of simplicity, we focus on the large-field setting throughout this work.

Our focus. Given the current landscape of VOLE-based ZK over batched disjunctive statements, a natural interesting question arises:

*Can we design a VOLE-based ZK protocol for batched disjunctive statements that improves on both **Batchman** [YHH⁺23a] and **Antman** [WYY⁺22] in communication, using only VOLE and AHE?*

1.1 Our Result

We answer the above questions affirmatively. Specifically, we construct a novel VOLE-based ZK protocol **Justvengers** — for batched disjunctive statements — by combining the techniques from **Antman** and **Batchman** in a non-trivial way.²

Table 1 summarizes **Justvengers**, compared to prior work. We highlight:

¹ This can be done even more efficiently using the *subfield* VOLE (e.g., [WYY⁺22]).

² **Bat[ch]man** hails from the “DC universe,” while **Antman** belongs to “Marvel” — together, they unite to form the “Justice Revengers.”

- Justvengers achieves $\mathcal{O}(R + B + C)$ communication, improving over both Batchman and Antman.
- Justvengers relies solely on VOLE and AHE, aligning with the cryptographic assumptions of Antman. (A RO can be used for concrete optimizations.)
- $\mathcal{V}$'s computation in Justvengers matches that of Antman and is incomparable to that of Batchman.
- $\mathcal{P}$'s computation in Justvengers is better than in Antman but worse than in Batchman. Crucially, in most parameter regimes, the complexity — $\tilde{\mathcal{O}}(BC + RB + RC)$ — is sublinear in the size of the “flattening” circuit representation.
- A batched disjunctive statement becomes uninformative if the witnesses across repetitions are uncorrelated, i.e., reusing the same witness across all R repetitions proves nothing. In Section 4.3, we show how Justvengers can enforce the correlation between witnesses at different repetitions.
- Justvengers, same as Antman, needs an AHE with *linear targeted malleability*.

Intuition: a bird’s eye view. We offer a brief yet intuitive explanation of how Justvengers achieves this. For a concise technical overview, see Section 3.

We begin by briefly reviewing the six core steps of Batchman [YHH⁺23a]: (1) $\mathcal{P}$ commits to the extended witness for each repetition, (2) $\mathcal{P}$ proves the well-formedness of all multiplications in R committed extended witnesses, (3) parties compress each branch into a length- $\mathcal{O}(C)$ vector called a *topology vector*, (4) $\mathcal{P}$ commits to the active topology vector for each repetition, (5) $\mathcal{P}$ proves that the inner product between the committed witness and the corresponding committed topology vector is zero, and (6) $\mathcal{P}$ proves that each committed topology vector is one of the valid public vectors. See Section 2.5 and Appendix A.3 for full details.

Justvengers builds on a key observation: Steps (1), (2), (4), and (5) can be viewed as a SIMD-style circuit — the SI circuit performs $\mathcal{O}(C)$ multiplications and a length- $\mathcal{O}(C)$ inner product. Thus, we apply the Antman technique to replace individual commitments with polynomial ones, reducing communication.

However, using polynomial encodings (and commitments) introduces a new challenge in performing Step (6), which checks membership in the public topology vector set. We address this by applying a random linear combination of the encoded polynomials, enabling an efficient universal hashing-based check.

1.2 Related Work

VOLE-based ZK. With the seminal work of [BCGI18] enabling cheap generation of VOLE correlations, a rich line of work on VOLE-based ZKP protocols soon emerged [BBMH⁺21, BBMHS22, BCC⁺24, BMRS21, DEGL⁺23, DIO21, DILO22, HY24, LHLA25, LXY24a, LXY24b, WYKW21, YSWW21]. See [BDSW23] for a survey. VOLE-based ZK is simple, highly efficient, and (mostly) information-theoretic in the VOLE-hybrid model. Its favorable scalability makes it especially well-suited for applications involving large statements.

Consider a standard fan-in 2 circuit $\mathcal{C}$ over a field $\mathbb{F}$, consisting of C gates. State-of-the-art VOLE-based ZK protocols [DIO21, YSWW21], which are IT secure, incur only linear costs with small constant factors.

VOLE-based ZK proofs are not succinct for arbitrary circuits when relying solely on VOLE correlations. By incorporating AHE, recent works have achieved sublinear communication — $\mathcal{O}(C^{3/4})$ in [WYY+22] and $\mathcal{O}(C^{1/2})$ in [BCC+24].

Non-batched and batched ZK disjunctions. The study of ZKP protocols for disjunctive statements dates back to the 1990s, beginning with the work of Cramer et al. [CDS94]. This problem was later revisited and refined by Heath and Kolesnikov [HK20], who focused on optimizing ZKPs based on *garbled circuits* [JKO13, Yao86]. Their work introduced the idea of “stacking,” a technique that reuses transmitted cryptographic material from the active branch to process (and privately discard) inactive branches. This reduces communication to that of the largest single branch but still requires evaluating all branches.

Following [HK20], a growing body of research [BMRS21, GGHAK22, GHAK24, GHAKS23, HHK+24, YHH+23a, YHH+24] explores stacking-style ZKPs across various ZK techniques. Among these, the work of [YHH+23a] is particularly relevant to ours, as it explicitly improves VOLE-based ZK for batched disjunctions.

We also highlight [WYY+22], which improves VOLE-based ZK for SIMD circuits. This implicitly improves ZK for batched disjunctions.

The core contribution of our work can be viewed as a synthesis of ideas from both [WYY+22] and [YHH+23a], combining their techniques to achieve more efficient VOLE-based ZK for batched disjunctive statements.

Other related work. ZKP is a vast and rapidly evolving research area. Here, we highlight a few relevant developments beyond our focus.

Recent work [BBD+23] introduced the *VOLE-in-the-head* cryptographic compiler, which enables any ZK protocol that relies solely on VOLE correlations — though not applicable to *Justvengers* and *Antman* — to be transformed into a non-interactive and publicly verifiable protocol.

Beyond VOLE-based ZK, there is considerable interest in succinct ZK proofs. These constructions can achieve polylogarithmic (or even sub-polylogarithmic) communication and verification complexity for *general circuits*. From a theoretical perspective, they offer asymptotically stronger succinctness guarantees, sometimes relying only on seemingly weaker assumptions. However, this comes at the cost of heavy computational burden on $\mathcal{P}$.

In contrast, VOLE-based ZK protocols — including *Justvengers* — prioritize concrete efficiency. They provide lightweight computation for $\mathcal{P}$, yielding the fastest known end-to-end proof times. Notably, *Justvengers* achieves (almost) sublinear prover computation with respect to the circuit size. To the best of our knowledge, such efficiency is not achievable in the current succinct ZK landscape. We note that the commit-and-prove structure of *Justvengers* may enable adaptations to succinct settings, which we leave to future work.

2 Preliminaries

Notation. λ denotes the security parameter (e.g., 128). The prover is denoted by $\mathcal{P}$ (referred to using she/her), and the verifier by $\mathcal{V}$ (referred to using he/him). $x := y$ denotes that y is *assigned* to x . Sets are denoted by uppercase letters, and

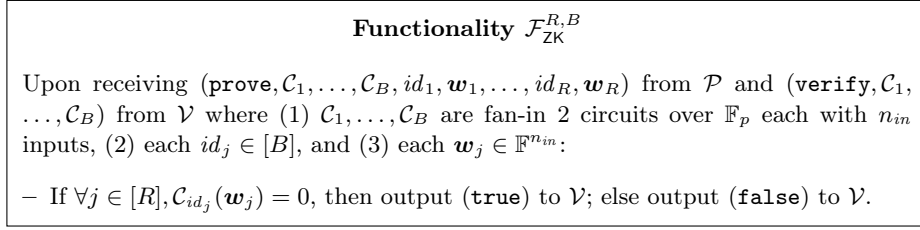


Fig. 1: The batched disjunctive ZK functionality.

$x \xleftarrow{\$} S$ indicates uniform sampling from a set S ; $[m]$ denotes the set $\{1, \dots, m\}$. We use $\mathbb{F}$ to represent a sufficiently large field, i.e., $|\mathbb{F}| = \lambda^{\omega(1)}$.³ Vectors are denoted by bold lowercase letters (e.g., $\mathbf{a}$), with a_i or $a[i]$ representing the i -th component (1-based). For vectors $\mathbf{a}$ and $\mathbf{b}$ of equal length, $\langle \mathbf{a}, \mathbf{b} \rangle$ denotes the inner product and $\mathbf{a} \odot \mathbf{b}$ the element-wise product. For a matrix M , $M_{i,j}$ refers to the entry in the i -th column and j -th row, and M_j denotes its j -th column. We use i to index branches ($i \in [B]$), j for repetitions ($j \in [R]$), k for gates ($k \in [n_{in} \text{ or } n_{\times}]$), and $id_j \in [B]$ to denote the active index of the j -th repetition.

2.1 Definitions Related to Polynomials

Definition 1 (Interpolation). Let $\alpha_1, \dots, \alpha_d \in \mathbb{F}$ be fixed, distinct public points. We define the procedure $\text{Inter}_{\alpha_1, \dots, \alpha_d}(y_1, \dots, y_d)$ to return the unique degree- $(d-1)$ (or a constant when $y_1 = \dots = y_d$) polynomial $f(X) \in \mathbb{F}[X]$ such that $f(\alpha_i) = y_i$ for all $i \in [d]$. This corresponds to standard Lagrange interpolation. When α is clear from context, we omit it for simplicity.

Definition 2 (Vanishing). Let $\alpha_1, \dots, \alpha_d \in \mathbb{F}$ be fixed, distinct public points. A polynomial $f(X) \in \mathbb{F}[X]$ is said to vanish at $\alpha_1, \dots, \alpha_d$ if and only if $f(\alpha_i) = 0$ for all $i \in [d]$. When α is clear from context, we omit it for simplicity.

The soundness of Justvengers relies on the standard *Schwartz-Zippel-DeMillo-Lipton* (SZDL) lemma [DL78, Sch80, Zip79], stated in Lemma 1.

Lemma 1 (Schwartz-Zippel-DeMillo-Lipton). Let $\mathbb{F}$ be a field and $p \in \mathbb{F}[x_1, \dots, x_n]$ be a (multivariate) polynomial of degree d . Suppose $|\mathbb{F}| > d$, then

$$\Pr \left[p(\mathbf{v}) = 0 \mid \mathbf{v} \xleftarrow{\$} \mathbb{F}^n \right] \leq \frac{d}{|\mathbb{F}|}$$

2.2 Security Model and Ideal Functionalities

We formalize our protocol within the UC framework [Can01], proving its security against a *malicious, static* adversary. For simplicity, we omit standard

³ Our protocol Justvengers assumes that each branch is expressed as an arithmetic circuit over $\mathbb{F}$, though it also supports small fields via field extensions.

Functionality $\mathcal{F}_{\text{Com}}$

This functionality initializes two empty set $S_{\mathcal{P}}, S_{\mathcal{V}}$ and runs with $\mathcal{P}$ and $\mathcal{V}$ as follows:

Commit. Upon receiving $(\text{commit}, m, \tau_m)$ from $\mathcal{P}$ (resp. $\mathcal{V}$) where there is no tuple $(\cdot, \tau_m)$ in $S_{\mathcal{P}}$ (resp. $S_{\mathcal{V}}$), store (m, τ_m) in $S_{\mathcal{P}}$ (resp. $S_{\mathcal{V}}$) and send τ_m to $\mathcal{V}$ (resp. $\mathcal{P}$). Here, τ_m is a handle that identifies the message m .

Open. Upon receiving (open, τ_m) from $\mathcal{P}$ (resp. $\mathcal{V}$) where there is a tuple (m, τ_m) in $S_{\mathcal{P}}$ (resp. $S_{\mathcal{V}}$), send (m, τ_m) to $\mathcal{V}$ (resp. $\mathcal{P}$).

Fig. 2: The standard two-party commitment functionality.

Functionality $\mathcal{F}_{\text{VOLE}}$

$\mathcal{F}_{\text{VOLE}}$, parameterized by a field $\mathbb{F}$, proceeds as follows, running with a prover $\mathcal{P}$, a verifier $\mathcal{V}$ and an adversary $\mathcal{S}$:

Initialize. Upon receiving (init) from $\mathcal{P}$ and $\mathcal{V}$, if $\mathcal{V}$ is honest, sample $\Delta \xleftarrow{\$} \mathbb{F}$, else receive Δ from $\mathcal{S}$. Store Δ and send it to $\mathcal{V}$. Ignore subsequent (init) .

Extend. Upon receiving (extend, n) from $\mathcal{P}$ and $\mathcal{V}$:

- If $\mathcal{V}$ is honest, sample $\mathbf{k}_u \xleftarrow{\$} \mathbb{F}^n$, else receive $\mathbf{k}_u \in \mathbb{F}^n$ from $\mathcal{S}$.
- If $\mathcal{P}$ is honest, sample $\mathbf{u} \xleftarrow{\$} \mathbb{F}^n$ and compute $\mathbf{m}_u := \mathbf{k}_u + \mathbf{u} \cdot \Delta \in \mathbb{F}^n$, else receive $\mathbf{u} \in \mathbb{F}^n$ and $\mathbf{m}_u \in \mathbb{F}^n$ from $\mathcal{S}$ and compute $\mathbf{k}_u := \mathbf{m}_u - \mathbf{u} \cdot \Delta \in \mathbb{F}^n$.
- Send $(\mathbf{u}, \mathbf{m}_u)$ to $\mathcal{P}$ and $\mathbf{k}_u$ to $\mathcal{V}$.

Fig. 3: The VOLE correlation functionality.

UC (sub-)session IDs. We also omit that each time a party sends a message to the functionality, it provides a receipt to the simulator, and that whenever the functionality needs to deliver an output to a party, it waits for the simulator's instruction to do so. Note, this includes the standard *security with abort*.

The ideal functionality achieved by **Justvengers** is defined in Figure 1. A standard commitment functionality, used by **Justvengers**, is defined in Figure 2.

2.3 IT-MAC and VOLE

IT-MAC. *Information-theoretic message authentication codes* (IT-MACs) — named by [BDOZ11, NNOB12] — are a form of distributed correlated randomness between two parties (in our case, $\mathcal{P}$ and $\mathcal{V}$) that can be used as commitments.

In IT-MACs over $\mathbb{F}$, $\mathcal{V}$ holds a uniformly sampled *global* key $\Delta \xleftarrow{\$} \mathbb{F}$. To commit to a value $x \in \mathbb{F}$, $\mathcal{V}$ samples a uniform *local* key $k_x \xleftarrow{\$} \mathbb{F}$, and $\mathcal{P}$ receives the corresponding MAC, computed as $m_x := k_x + x\Delta$. We denote the IT-MAC cor-

relation of a value x as $[x]_\Delta := \langle (x, m_x), k_x \rangle_\Delta$. When clear from context, we omit Δ for simplicity. Below, we summarize several useful properties of IT-MACs:

1. **Hiding:** k_x and Δ , held by $\mathcal{V}$, are independent of the committed value x .
2. **Binding:** To open $[x]$, $\mathcal{P}$ sends x and m_x , and $\mathcal{V}$ accepts if $m_x = k_x + x\Delta$. To maliciously open $[x]$ as a different value $x' \neq x$ (i.e., forge x), $\mathcal{P}$ would need to guess Δ , succeeding with probability at most $\frac{1}{|\mathbb{F}|}$.
3. **Linear Homomorphism:** IT-MACs support linear operations — such as addition, scalar multiplication, and constant addition — *without* any communication. Specifically, for any *public* constants $c_0, c_1, \dots, c_n \in \mathbb{F}$, the parties can *locally* compute $[c_0 + c_1x_1 + \dots + c_nx_n]$ from $[x_1], \dots, [x_n]$.⁴ We denote $[c_0 + c_1x_1 + \dots + c_nx_n] = c_0 + c_1 \cdot [x_1] + \dots + c_n \cdot [x_n]$. Note, this implies that an IT-MAC of a public constant can be generated *for free*.

VOLE. Random IT-MAC instances (over $\mathbb{F}$) can be viewed as the standard *vector oblivious linear evaluation* (VOLE) correlation functionality, defined as $\mathcal{F}_{\text{VOLE}}$ in Figure 3. This functionality has been extensively studied in prior work (e.g., [BCGI18, BCG⁺19, SGRR19, YWL⁺20]). It can be realized efficiently, achieving even sublinear communication in n (e.g., [BCGI18]) with the *learning parity with noise* (LPN) assumption [BFKL94].

Lemma 2 (Informal). *Assuming the hardness of LPN, there exists a protocol that UC-emulates $\mathcal{F}_{\text{VOLE}}$ (Figure 3) with $o(n)$ communication for each extend.*

It is standard [Bea95] to *consume* a random IT-MAC $[u]$ to generate an IT-MAC $[x]$ for a value x chosen by $\mathcal{P}$: (1) $\mathcal{P}$ sends $x - u$, and (2) parties *locally* compute $[x] := [u] + (x - u)$. Note that each random u can only be used once.

2.4 IT-PAC

The core technique introduced by Antman [WYY⁺22] (the protocol is reviewed in Section 2.5) is a novel method for committing to polynomials using IT-MACs. Specifically, $\mathcal{V}$ additionally holds a *privately* sampled global evaluation point $\Lambda \xleftarrow{\$} \mathbb{F}$. To commit to a polynomial $f(\cdot) \in \mathbb{F}[X]$ held by $\mathcal{P}$, parties obtain $[f(\Lambda)]$, i.e., the IT-MAC for the value $f(\Lambda)$. Clearly, we have the following properties:

1. **Hiding:** $[f(\Lambda)]$ reveals no information about the polynomial $f(\cdot)$ to $\mathcal{V}$.
2. **Binding:** To open the committed $f(\cdot)$, $\mathcal{P}$ sends $f(\cdot)$ and $m_{f(\Lambda)}$, and $\mathcal{V}$ accepts if $m_{f(\Lambda)} = k_{f(\Lambda)} + f(\Lambda)\Delta$. To forge the commitment to a different polynomial $f'(\cdot)$, $\mathcal{P}$ must either (1) guess Δ , which succeeds with probability at most $\frac{1}{|\mathbb{F}|}$, or (2) ensure that $f(\Lambda) = f'(\Lambda)$, which succeeds with probability at most $\frac{d}{|\mathbb{F}|}$, where d is the maximum degree of $f(\cdot)$ and $f'(\cdot)$ by Lemma 1.
3. **Linear Homomorphism:** $[c_0 + c_1f_1(\Lambda) + \dots + c_nf_n(\Lambda)] = c_0 + c_1 \cdot [f_1(\Lambda)] + \dots + c_n \cdot [f_n(\Lambda)]$ for any *public* constants $c_0, c_1, \dots, c_n \in \mathbb{F}$.

In Antman [WYY⁺22], this form of correlation — commitments to polynomials — is referred to as the *information-theoretic polynomial authentication code* (IT-PAC). We make the following remarks regarding IT-PACs:

⁴ For example, if $m_x = k_x + x\Delta$ and $m_y = k_y + y\Delta$, then $(m_x + m_y) = (k_x + k_y) + (x + y)\Delta$. Moreover, for any constant $c \in \mathbb{F}$, $\mathcal{P}$ sets $m_c = 0$ whereas $\mathcal{V}$ sets $k_c = -c\Delta$.

Remark 1. The binding property of an IT-PAC $[f(\Lambda)]$ relies on the fact that the prover $\mathcal{P}$ has *no knowledge* of Λ , including the value $f(\Lambda)$. This contrasts with our notation for commitments like $[x]$ as IT-MACs, where the committed value x is known to $\mathcal{P}$. To emphasize this distinction, we use the notation $[f(\cdot)]$ to represent the IT-PAC of the polynomial $f(\cdot)$.

Remark 2. Looking ahead, $\mathcal{V}$ would reveal Λ to $\mathcal{P}$ at some step in Justvengers (as in Antman; see Section 2.5). To preserve soundness, we require $\mathcal{P}$ to commit to any polynomial to be opened (e.g., $f(\cdot)$) using the standard commitment functionality $\mathcal{F}_{\text{Com}}$ (Figure 2), *before* learning Λ . Once $\mathcal{P}$ learns Λ , the IT-PAC $[f(\cdot)]$ becomes a standard IT-MAC $[f(\Lambda)]$. Crucially, it is insecure for $\mathcal{P}$ to open any committed polynomials *after* learning Λ .

Remark 3. IT-PACs are weaker than traditional polynomial commitment schemes commonly used in the SNARK literature, e.g., [KZG10]. Notably, IT-PACs do *not* support evaluating the committed polynomial at arbitrary points.

Generating IT-PACs from AHE. IT-PACs can be efficiently generated using *additively homomorphic encryption* (AHE) combined with VOLE correlations, as demonstrated in [WYY⁺22]. To support degree- $\leq d$ polynomials, $\mathcal{V}$ sends d encrypted powers of Λ to $\mathcal{P}$, i.e., $\llbracket \Lambda \rrbracket, \llbracket \Lambda^2 \rrbracket, \dots, \llbracket \Lambda^d \rrbracket$, where $\llbracket m \rrbracket = \text{Enc}(\text{sk}, m; r)$ denotes an AHE encryption of m . Given these ciphertexts, a randomly sampled IT-MAC $[u]$, and a degree- d polynomial $f(\cdot)$ of $\mathcal{P}$'s choice, $\mathcal{P}$ can use the additive homomorphism to compute $\llbracket f(\Lambda) - u \rrbracket$, which will be sent to $\mathcal{V}$. Upon decryption, $\mathcal{V}$ learns $f(\Lambda) - u$, which effectively acts as a one-time pad on $f(\Lambda)$.

Note that $[f(\Lambda) - u]$ can be locally generated with only $\mathcal{V}$'s knowing $f(\Lambda) - u$.⁵ Thus, parties can obtain $[f(\Lambda)] := [u] + [f(\Lambda) - u]$.

As noted by [WYY⁺22], this requires an AHE scheme that satisfies the standard *chosen plaintext attack* (CPA) security, and achieves the *circuit privacy* [IP07] and *linear targeted malleability* [BCI⁺22]. In particular, this AHE can be instantiated by the standard BGV-AHE scheme [BGV12], as analyzed in [KPR18, WYY⁺22]. We defer further discussion in Appendix A.1.

We note that the above method only ensures security against a semi-honest $\mathcal{V}$. However, it already suffices for Justvengers (and Antman [WYY⁺22]). Looking ahead, $\mathcal{P}$ will eventually learn Λ and can safely learn the randomness used in the AHE encryptions. Thus, $\mathcal{P}$ can commit to the messages she intends to send (e.g., $\llbracket f(\Lambda) - u \rrbracket$) and disclose them only after verifying the correctness of the encrypted values of Λ and its powers. This elegant approach, used by Antman [WYY⁺22], can be traced back to GC-ZK [JKO13].

Finally, note that the d AHE encryptions can be reused to generate IT-PACs for multiple degree- $\leq d$ polynomials. This leads to the following:

Lemma 3 (Informal). *If AHE is an AHE scheme that satisfies CPA security, circuit privacy, and linear targeted malleability, then there exists a protocol for generating n IT-PACs over polynomials of degree at most d , communicating only $O(n + d)$ AHE ciphertexts in the $\mathcal{F}_{\text{VOLE}}$ -hybrid model (Figure 3).*

⁵ $\mathcal{P}$ sets $m_{f(\Lambda)-u} = 0$ whereas $\mathcal{V}$ sets $k_{f(\Lambda)-u} = -(f(\Lambda) - u)\Delta$. We slightly abuse the IT-MAC notation by allowing $\mathcal{P}$ to hold a committed value she does *not* know.

2.5 VOLE-Based ZK

In this section, we briefly review several VOLE-based ZK protocols, focusing in particular on Antman [WYY⁺22] and Batchman [YHH⁺23a], which are crucial to understand our novel protocol Justvengers.

LPZK [DIO21, YSWW21]. VOLE-based ZK follows the typical “commit-and-prove” paradigm: $\mathcal{P}$ commits to her extended witness with IT-MACs, and later proves to $\mathcal{V}$ that committed values are consistent with the proof statement.

Given a circuit $\mathcal{C}$ (over $\mathbb{F}$), $\mathcal{P}$ commits to its inputs and the outputs of multiplication gates. Due to the linear homomorphism, parties can locally derive IT-MACs for all wires in $\mathcal{C}$. The remaining task is for $\mathcal{P}$ to prove:

1. **Satisfied Circuit:** The IT-MAC on the output wire of $\mathcal{C}$ commits to a zero. This can be proved by simply opening the IT-MAC.
2. **Correct Multiplication:** Each multiplication gate in $\mathcal{C}$ must be well-formed. For the k -th multiplication gate, parties hold three IT-MACs $[\ell_k], [r_k], [o_k]$ corresponding to the left input, right input, and output wires. $\mathcal{P}$ must prove in ZK that $\ell_k \cdot r_k = o_k$.

The LPZK technique [DIO21, YSWW21] enables this proof to be performed over n such triples using only $\mathcal{O}(1)$ communication by using an extra VOLE correlation. We provide a consise review of LPZK in Appendix A.2.

Although the LPZK technique achieves constant communication overhead for verifying multiple MULT triples, $\mathcal{P}$ still needs to commit to the (extended) witness, incurring communication that scales linearly with the size of the circuit.

Antman [WYY⁺22]. Prior work has shown that communication costs can be reduced for circuits with special structures. For instance, Antman [WYY⁺22] optimizes VOLE-based ZK protocols for *single instruction multiple data* (SIMD) circuits, where the same circuit is proven R times with potentially different inputs. This optimization is directly applicable to our setting involving batched disjunctive statements, where the “single instruction” (SI) corresponds to a multiplexer spanning all branches — resulting in an SI circuit of size $\mathcal{O}(BC)$.

The core insight of Antman is that $\mathcal{P}$ can encode values on the same wire across R repetitions as a single polynomial. Specifically, $\mathcal{P}$ and $\mathcal{V}$ agree on R fixed, distinct public points $\alpha_1, \dots, \alpha_R \in \mathbb{F}$. For the k -th input wire, $\mathcal{P}$ ’s witness defines R values across the R repetitions, denoted $in_k^{(1)}, \dots, in_k^{(R)}$. $\mathcal{P}$ constructs the degree- $(R-1)$ polynomial $in_k(X) := \text{Inter}(in_k^{(1)}, \dots, in_k^{(R)}) \in \mathbb{F}[X]$, and commits to it using an IT-PAC, denoted $[in_k(\cdot)]$. The same approach applies to the output of each multiplication gate, which is also encoded and committed as a degree- $(R-1)$ polynomial. This polynomial-based encoding allows $\mathcal{P}$ to commit across all repetitions using only $\mathcal{O}(BC + R)$ communication; see Lemma 3.

Due to the linear homomorphism, parties can locally derive IT-PACs for all wires in the SI circuit. The remaining task is for $\mathcal{P}$ to prove:

1. **Satisfied Circuit:** The IT-PAC on the output wire of the SI circuit commits to a vanishing polynomial at α . Note that since the polynomial is of degree $(R-1)$ and vanishes at R distinct points, it must be the zero polynomial — the IT-PAC reduces to an IT-MAC of zero, which $\mathcal{P}$ can simply open to $\mathcal{V}$.

2. **Correct Multiplication:** Each multiplication gate in the SI circuit must be well-formed. For the k -th multiplication gate, parties hold three IT-PACs $[\ell_k(\cdot)], [r_k(\cdot)], [o_k(\cdot)]$ corresponding to the left input, right input, and output wires. $\mathcal{P}$ must prove in ZK that $\forall j \in [R], \ell_k(\alpha_j) \cdot r_k(\alpha_j) = o_k(\alpha_j)$.

This can be done as follows:

- (a) $\mathcal{P}$ computes the degree- $(2R-2)$ polynomial $\tilde{o}_k(X) := \ell_k(X) \cdot r_k(X)$, and commits to it using an IT-PAC as $[\tilde{o}_k(\cdot)]$.
- (b) Parties locally compute IT-PAC $[\tilde{o}_k(\cdot) - o_k(\cdot)]$, which, by construction, must be a vanishing polynomial in a valid proof. To prove this in ZK, $\mathcal{P}$ commits to a uniformly random vanishing polynomial $r(X)$ of degree $(2R-2)$ as $[r(\cdot)]$, and then opens the IT-PAC $[\chi \tilde{o}_k(\cdot) - \chi o_k(\cdot) + r(\cdot)]$, where $\chi \in \mathbb{F}$ is a random challenge selected and sent by $\mathcal{V}$. Finally, $\mathcal{V}$ checks whether the revealed polynomial is indeed vanishing.

Here, $r(X)$ is a one-time polynomial pad to preserve ZK. This approach naturally generalizes: to prove that n IT-PACs $[f_1(\cdot)], \dots, [f_n(\cdot)]$ are all vanishing, $\mathcal{P}$ opens a single IT-PAC of the form $[\sum_{i=1}^n \chi^i f_i(\cdot) + r(\cdot)]$, and $\mathcal{V}$ verifies that the resulting polynomial is vanishing.⁶

- (c) $\mathcal{V}$ reveals Λ to $\mathcal{P}$. At this point, for each k , IT-PACs $[\ell_k(\cdot)], [r_k(\cdot)], [\tilde{o}_k(\cdot)]$ become IT-MACs $[\ell_k(\Lambda)], [r_k(\Lambda)], [\tilde{o}_k(\Lambda)]$. Parties then invokes a standard LPZK protocol to prove that $\ell_k(\Lambda) \cdot r_k(\Lambda) = \tilde{o}_k(\Lambda)$ for all k , ensuring that each $\tilde{o}_k(\cdot)$ was correctly constructed and committed.

We emphasize that Λ must only be disclosed after $\mathcal{P}$ has fully opened all committed polynomials via IT-PACs. This is achievable as these messages sent by $\mathcal{P}$ are independent of Λ .

Batchman [YHH⁺23a]. In contrast to Antman, the VOLE-based ZK protocol Batchman [YHH⁺23a] is restricted to batched disjunctive statements and relies solely on IT-MACs. Recall that in a batched disjunctive statement, $\mathcal{P}$ and $\mathcal{V}$ agree on B fan-in 2 circuits $\mathcal{C}_1, \dots, \mathcal{C}_B$ over $\mathbb{F}$. W.l.o.g., assume that each circuit is of size C with n_{in} inputs and $n_{\times}$ multiplications. $\mathcal{P}$'s goal is to demonstrate the knowledge of R witnesses ($id_j \in [B]$, $\mathbf{w}_j \in \mathbb{F}^{n_{in}}$) for each $j \in [R]$ s.t. $\forall j \in [R], \mathcal{C}_{id_j}(\mathbf{w}_j) = 0$ where neither $\mathbf{w}_j$ nor id_j is revealed.

To illustrate the basic idea behind Batchman, consider the special case where $R = 1$ and $\mathcal{P}$ holds a valid witness $\mathbf{w}$ on $\mathcal{C}_{id}$. In Batchman, $\mathcal{P}$ commits to her extended witness *only* for the active (i.e., id -th) branch. Specifically, $\mathcal{P}$ commits to the n_{in} input wires, as well as the left and right inputs and the outputs of the $n_{\times}$ multiplication gates, defined by $\mathcal{C}_{id}(\mathbf{w})$. That is, parties obtain:

$$[\mathbf{w}_{ext}] := [in_1], \dots, [in_{n_{in}}], [\ell_1], \dots, [\ell_{n_{\times}}], [r_1], \dots, [r_{n_{\times}}], [o_1], \dots, [o_{n_{\times}}]$$

Parties then invoke LPZK to verify that $\ell \odot \mathbf{r} = \mathbf{o}$, ensuring that all multiplication constraints are satisfied. Crucially, the total number of IT-MACs is $\mathcal{O}(C)$, and $\mathcal{V}$ cannot distinguish which branch $\mathcal{P}$ intends to prove. Intuitively, it now suffices

⁶ We note that this generalized technique is crucial for efficiency. Opening a degree- $\mathcal{O}(R)$ polynomial requires transmitting $\mathcal{O}(R)$ coefficients; thus, naively opening n such randomized polynomials would incur $\mathcal{O}(nR)$ communication. On the other hand, this technique reduces the total communication to just $\mathcal{O}(R)$.

for $\mathcal{P}$ to show that the committed extended witness satisfies a specific linear relation corresponding to one of the branches — without revealing which one.

This is accomplished using the concept called *topology vector*. Specifically, each branch circuit $\mathcal{C}_i$ induces a public (randomized) vector $\mathbf{tv}_i$ of length $n_{in} + 3n_{\times} + 1$. The i -th branch is the active one *if and only if* the inner product $\langle \mathbf{w}_{ext} \| 1, \mathbf{tv}_i \rangle = 0$.⁷ Naïvely checking all topology vectors via inner products would still incur $\mathcal{O}(BC)$ overhead. Instead, $\mathcal{P}$ additionally commits to the topology vector $\mathbf{tv}_{id}$ corresponding to the active branch, maintaining overall communication of $\mathcal{O}(C)$. Finally, $\mathcal{P}$ must prove that the committed topology vector is indeed one of the B publicly known vectors. This membership check can be performed efficiently using standard universal hashing techniques.

Casting this to more repetitions, the key observation is that the same topology vectors can be reused across all repetitions. The overall **Batchman** protocol consists of the following six core steps:

1. $\mathcal{P}$ commits to the extended witness for each of the R repetitions.
2. $\mathcal{P}$ proves in ZK that all multiplications are formed correctly.
3. Parties get the (randomized) topology vectors corresponding to each branch.
4. $\mathcal{P}$ commits to the topology vector for each of the R repetitions.
5. $\mathcal{P}$ proves in ZK that the inner product between each committed extended witness and its corresponding committed topology vector equals zero.
6. $\mathcal{P}$ proves that each committed topology vector belongs to the set of B publicly known topology vectors.

Clearly, committing to the extended witness across R repetitions already incurs $\mathcal{O}(RC)$ communication. Looking ahead, **Justvengers** implements each of these steps using IT-PACs, reducing the communication overhead.

For simplicity, we omit some details (e.g., the topology vector) and defer a more comprehensive discussion of **Batchman** to Appendix A.3.

3 Technical Overview

In this section, we provide a technical overview of **Justvengers** to highlight our core contribution. We note that understanding how **Antman** [WYY⁺22] and **Batchman** [YHH⁺23a] work (see Section 2.5 for a concise review) would be very helpful to contextualize the components in this section. We refer the reader to Section 1.1 for a high-level intuition behind **Justvengers**.

3.1 Operations Enabled by IT-PACs

In this section, we abstract out several operations on (committed) polynomials enabled by IT-PACs (see Sections 2.4 and 2.5). We remark that these operations are significantly more complex when formally implemented — for example, involving the “commit-disclose-open” technique (as in GC-ZK [JKO13]) and the use of AHE to instantiate IT-PACs. However, these underlying details can be omitted for understanding the construction of **Justvengers**.

⁷ The appended 1 accounts for constant-related gates; see Figure 10 in Appendix A.3.

Recall that IT-PACs require specifying an upper bound on the degree of polynomials that $\mathcal{P}$ may commit to. In our setting, we fix this maximum degree to be $2R-2$, where R is the number of repetitions. Additionally, $\mathcal{P}$ and $\mathcal{V}$ agree on R fixed, distinct public points $\alpha_1, \dots, \alpha_R \in \mathbb{F}$.⁸

IT-PACs support the following operations:

- **Commit a Polynomial:** After a one-time initialization in which $\mathcal{V}$ sends $\mathcal{O}(R)$ communication (i.e., $\mathcal{O}(R)$ AHE ciphertexts over Λ and its powers), $\mathcal{P}$ can commit to each polynomial of degree at most $2R-2$ at a cost of $\mathcal{O}(1)$ communication and a single VOLE correlations per commitment. In Section 3.3, we omit the $\mathcal{O}(R)$ initialization cost for simplicity, i.e., the overall communication cost implicitly includes an additive $\mathcal{O}(R)$ term.
- **Multiply Two Polynomials:** Given IT-PACs $[f(\cdot)]$ and $[g(\cdot)]$, the parties can compute the IT-PAC $[f(\cdot)g(\cdot)]$ at a cost of $\mathcal{O}(1)$ communication and $\mathcal{O}(1)$ VOLE correlations per multiplication. One constraint is that the resulting product must have degree at most $2R-2$.
- **Prove Vanishing Polynomials:** Given n IT-PACs $[f_1(\cdot)], \dots, [f_n(\cdot)]$, $\mathcal{P}$ can prove in ZK that each committed polynomial vanishes at the points α with $\mathcal{O}(R)$ communication and $\mathcal{O}(1)$ VOLE correlations.

If an IT-PAC $[f(\cdot)]$ commits to a vanishing polynomial of degree $< R$, then $f(\cdot)$ must be a zero polynomial. In this case, $\mathcal{P}$ can simply open the IT-MAC corresponding to zero, incurring only $\mathcal{O}(1)$ communication per proof.⁹

In Section 3.3, whenever we refer to proving that a committed polynomial is vanishing, we implicitly mean that all such proofs are aggregated into a single batched proof (which costs $\mathcal{O}(R)$ communication in total).

3.2 Polynomial Interpolation over Matrices

In this section, we extend the procedure `Inter` for generating polynomials row-by-row from a matrix. Given an $n \times R$ matrix M , the procedure `Inter`(M) produces n polynomials of degree at most $(R-1)$ by performing interpolation over each row, using a fixed set of evaluation points α . This procedure is depicted as follows:

$$M = \begin{pmatrix} M_{1,1} & M_{2,1} & \cdots & M_{R,1} \\ M_{1,2} & M_{2,2} & \cdots & M_{R,2} \\ \cdots & \cdots & \cdots & \cdots \\ M_{1,n} & M_{2,n} & \cdots & M_{R,n} \end{pmatrix} \xrightarrow{\text{Inter}} \begin{array}{l} M_1(\cdot) := \text{Inter}_{\alpha_1, \dots, \alpha_R}(M_{1,1}, \dots, M_{R,1}) \\ M_2(\cdot) := \text{Inter}_{\alpha_1, \dots, \alpha_R}(M_{1,2}, \dots, M_{R,2}) \\ \cdots \\ M_n(\cdot) := \text{Inter}_{\alpha_1, \dots, \alpha_R}(M_{1,n}, \dots, M_{R,n}) \end{array}$$

The crucial observation is: Committing to each entry of the matrix M individually using IT-MACs requires $\mathcal{O}(nR)$ communication and VOLE correlations. In contrast, committing to each row as a degree- $\leq (R-1)$ polynomial using IT-PACs reduces the cost to just $\mathcal{O}(n)$ communication and VOLE correlations (and a one-time initialization overhead of $\mathcal{O}(R)$ communication).

⁸ In particular, choosing NTT-friendly points can significantly accelerate computation.

⁹ By using the random oracle, n such zero IT-MAC proofs can be compressed into $\mathcal{O}(\lambda)$ total communication in bits; see Section 4.2.

3.3 Justvengers: A Step-By-Step Overview

Recall that **Justvengers** is constructed by replacing each step of **Batchman** with IT-PAC-based constructions to reduce communication. In particular, these replacements leverage the operations introduced in Section 3.1.

In this section, we revisit the six core steps of **Batchman** outlined in Section 2.5 and show how each is adapted in **Justvengers**. Alongside each modification, we highlight the corresponding communication improvements achieved by transitioning from IT-MACs to IT-PACs.

Batched disjunctive statements. As a reminder, **Justvengers** is designed specifically for handling batched disjunctive statements. In such a statement, $\mathcal{P}$ and $\mathcal{V}$ agree on B fan-in 2 circuits $\mathcal{C}_1, \dots, \mathcal{C}_B$ over $\mathbb{F}$. W.l.o.g., assume that each circuit is of size C with n_{in} inputs and $n_{\times}$ multiplications. $\mathcal{P}$'s goal is to demonstrate the knowledge of R witnesses ($id_j \in [B]$, $\mathbf{w}_j \in \mathbb{F}^{n_{in}}$) for each $j \in [R]$ s.t. $\forall j \in [R], \mathcal{C}_{id_j}(\mathbf{w}_j) = 0$ where neither $\mathbf{w}_j$ nor id_j is revealed.

Step 1: $\mathcal{P}$ commits to inputs and multiplication triples. Recall that in **Batchman**, for the j -th repetition, $\mathcal{P}$ commits — using IT-MACs — to the inputs and multiplication triples of the active branch, denoted as the extended witness. These committed values can be organized into a $(n_{in} + 3n_{\times}) \times R$ matrix W , where the j -th column corresponds to the j -th extended witness.

In **Justvengers**, $\mathcal{P}$ applies the **Inter** procedure to W (see Section 3.2) — interpolating each row into a degree- $< R$ polynomial — and then commits to $\mathcal{O}(C)$ resulting polynomials using IT-PACs. This process is depicted as follows:

$$W = \begin{pmatrix} in_1^{(1)} & in_1^{(2)} & \dots & in_1^{(R)} \\ \dots & \dots & \dots & \dots \\ in_{n_{in}}^{(1)} & in_{n_{in}}^{(2)} & \dots & in_{n_{in}}^{(R)} \\ \ell_1^{(1)} & \ell_1^{(2)} & \dots & \ell_1^{(R)} \\ \dots & \dots & \dots & \dots \\ \ell_{n_{\times}}^{(1)} & \ell_{n_{\times}}^{(2)} & \dots & \ell_{n_{\times}}^{(R)} \\ r_1^{(1)} & r_1^{(2)} & \dots & r_1^{(R)} \\ \dots & \dots & \dots & \dots \\ r_{n_{\times}}^{(1)} & r_{n_{\times}}^{(2)} & \dots & r_{n_{\times}}^{(R)} \\ o_1^{(1)} & o_1^{(2)} & \dots & o_1^{(R)} \\ \dots & \dots & \dots & \dots \\ o_{n_{\times}}^{(1)} & o_{n_{\times}}^{(2)} & \dots & o_{n_{\times}}^{(R)} \end{pmatrix} \xrightarrow{\text{Inter}} \begin{matrix} IN_1(\cdot) \\ \dots \\ IN_{n_{in}}(\cdot) \\ L_1(\cdot) \\ \dots \\ L_{n_{\times}}(\cdot) \\ R_1(\cdot) \\ \dots \\ R_{n_{\times}}(\cdot) \\ O_1(\cdot) \\ \dots \\ O_{n_{\times}}(\cdot) \end{matrix} \xrightarrow{\text{Commit}} \begin{matrix} [IN_1(\cdot)] \\ \dots \\ [IN_{n_{in}}(\cdot)] \\ [L_1(\cdot)] \\ \dots \\ [L_{n_{\times}}(\cdot)] \\ [R_1(\cdot)] \\ \dots \\ [R_{n_{\times}}(\cdot)] \\ [O_1(\cdot)] \\ \dots \\ [O_{n_{\times}}(\cdot)] \end{matrix}$$

Here, $in_k^{(j)}$ denotes the k -th input in the j -th repetition for $j \in [R]$ and $k \in [n_{in}]$. The notation is analogous for $\ell_k^{(j)}$, $r_k^{(j)}$, and $o_k^{(j)}$ for $k \in [n_{\times}]$, representing the corresponding multiplication gate wires. On the other hand, $IN_k(\cdot)$ denotes the polynomial obtained by interpolating the k -th input values across the R repetitions, for $k \in [n_{in}]$. Namely, $IN_k(\alpha_j) = in_k^{(j)}$ for all $j \in [R]$. The same notation

applies to $L_k(\cdot)$, $R_k(\cdot)$, and $O_k(\cdot)$ for $k \in [n_\times]$, representing the interpolated polynomials corresponding to the multiplication gate wires.

Cost: This step in **Justvengers** requires $\mathcal{O}(C)$ communication and $\mathcal{O}(C)$ VOLE correlations, improving over the corresponding step in **Batchman**, which incurs $\mathcal{O}(RC)$ communication and VOLE correlations.

Step 2: $\mathcal{P}$ proves well-formed multiplications. At this stage, $\mathcal{P}$ has encoded her extended witness using $\mathcal{O}(C)$ polynomials and committed to them via IT-PACs. To ensure correctness, the parties must verify that the multiplication gates are properly encoded within these polynomials. Formally, $\mathcal{P}$ must prove in ZK that:

$$\forall k \in [n_\times], \forall j \in [R], L_k(\alpha_j) \cdot R_k(\alpha_j) = O_k(\alpha_j)$$

Recall that **Antman** includes a sub-protocol designed explicitly for this purpose, which we directly incorporate into **Justvengers**. For each $k \in [n_\times]$, the parties (1) generate the IT-PAC $[L_k(\cdot)R_k(\cdot)]$ from the existing IT-PACs $[L_k(\cdot)]$ and $[R_k(\cdot)]$; and (2) prove that the IT-PAC $[L_k(\cdot)R_k(\cdot) - O_k(\cdot)]$ commits a vanishing polynomial. (Note that the subtraction is a free linear operation.)

Cost: This step in **Justvengers** requires $\mathcal{O}(C)$ communication (plus a one-time $\mathcal{O}(R)$ cost for batching the vanishing polynomial proofs) and $\mathcal{O}(C)$ VOLE correlations. While the corresponding step in **Batchman** incurs only $\mathcal{O}(1)$ communication, this additional overhead does *not* affect the overall asymptotic.

Step 3: Parties generate topology vectors for each branch. This step in **Justvengers** matches that of **Batchman** (see Appendix A.3) and requires only $\mathcal{O}(1)$ communication. We denote these B length- $\mathcal{O}(C)$ vectors as $\mathbf{tv}^{(1)}, \dots, \mathbf{tv}^{(B)}$.

Step 4: $\mathcal{P}$ commits to active topology vectors. $\mathcal{P}$ must convince $\mathcal{V}$ that each column of W corresponds to a valid branch — that is, there exists a topology vector (among the B candidates) whose inner product with the column yields 0.

In **Batchman**, this is accomplished by having $\mathcal{P}$ commit — using IT-MACs — to the active topology vector used in each repetition. Similar to Step 1, **Justvengers** arranges the R active topology vectors column-wise — one per repetition — into a $(n_{in} + 3n_\times + 1) \times R$ matrix, denoted TV . $\mathcal{P}$ then applies interpolation row-by-row and commits to the resulting polynomials using IT-PACs.

For each repetition $j \in [R]$, let $id_j \in [B]$ denote the index of the active branch. This process is depicted as follows:

$$TV = \left(\mathbf{tv}^{(id_1)} \dots \mathbf{tv}^{(id_R)} \right) \xrightarrow{\text{Inter}} \begin{matrix} TV_1(\cdot) \\ \dots \\ TV_{n_{in}+3n_\times+1}(\cdot) \end{matrix} \xrightarrow{\text{Commit}} \begin{matrix} [TV_1(\cdot)] \\ \dots \\ [TV_{n_{in}+3n_\times+1}(\cdot)] \end{matrix}$$

Here, for each $k \in [n_{in} + 3n_\times + 1]$, the polynomial $TV_k(\cdot)$ encodes the k -th entry of the active topology vector across all R repetitions.

Cost: This step in **Justvengers** requires $\mathcal{O}(C)$ communication and $\mathcal{O}(C)$ VOLE correlations, improving over the corresponding step in **Batchman**, which incurs $\mathcal{O}(RC)$ communication and VOLE correlations.

Step 5: $\mathcal{P}$ proves zero inner products. $\mathcal{P}$ must prove in ZK that the inner product between each committed extended witness and its corresponding committed topology vector equals zero. That is, for each $j \in [R]$, $\mathcal{P}$ must show that the inner product of W_j and TV_j is zero. Formally, this means: $\forall j \in [R]$,

$$\langle (\{IN_k(\alpha_j)\}_{k \in [n_{in}]}, \{L_k(\alpha_j)\}_{k \in [n_{\times}]}, \{R_k(\alpha_j)\}_{k \in [n_{in}]}, \{O_k(\alpha_j)\}_{k \in [n_{in}]}, 1), (\{TV_k(\alpha_j)\}_{k \in [n_{in}+3n_{\times}+1]}) \rangle = 0$$

Here, the constant term “1” is appended to account for constant-related gates.¹⁰ Let $UNIT(\cdot) = 1$ denote the constant polynomial that evaluates to 1 everywhere. Note that the IT-PAC for $UNIT(\cdot)$ can be generated *for free*, as $[UNIT(\cdot)] = [1]$.

To finish this step, the core observation made by **Justvengers** is that parties can generate the following IT-PAC, denoted $[IP(\cdot)]$:

$$\begin{aligned} & \left[\sum_{k=1}^{n_{in}} IN_k(\cdot) TV_k(\cdot) + \sum_{k=1}^{n_{\times}} L_k(\cdot) TV_{n_{in}+k}(\cdot) + \sum_{k=1}^{n_{\times}} R_k(\cdot) TV_{n_{in}+n_{\times}+k}(\cdot) \right. \\ & \quad \left. + \sum_{k=1}^{n_{\times}} O_k(\cdot) TV_{n_{in}+2n_{\times}+k}(\cdot) + UNIT(\cdot) TV_{n_{in}+3n_{\times}+1}(\cdot) \right] \end{aligned}$$

Specifically, this polynomial can be constructed by multiplying $\mathcal{O}(C)$ pairs of IT-PACs. By construction, for each $j \in [R]$, the evaluation $IP(\alpha_j)$ corresponds to the inner product for the j -th repetition. Thus, it suffices for $\mathcal{P}$ to prove that the committed polynomial $IP(\cdot)$ is vanishing.

Cost: This step in **Justvengers** requires $\mathcal{O}(C)$ communication (plus a one-time $\mathcal{O}(R)$ cost for batching the vanishing polynomial proofs) and $\mathcal{O}(C)$ VOLE correlations. While the corresponding step in **Batchman** incurs only $\mathcal{O}(1)$ communication, this additional overhead does *not* affect the overall asymptotic.

Step 6: $\mathcal{P}$ proves membership of committed topology vectors. $\mathcal{P}$ must prove that each committed active topology vector — encoded within polynomials — is indeed one of the B publicly known topology vectors. To illustrate how this is achieved in **Justvengers**, we first recall how membership checks over committed vectors can be performed efficiently, as done in **Batchman**.

The core idea relies on universal hashing. Suppose $\mathcal{P}$ holds a committed vector $[v]$ (using IT-MACs) and needs to prove that v is contained in the public set of topology vectors. It suffices for $\mathcal{V}$ to sample a random challenge $\gamma \xleftarrow{\$} \mathbb{F}$, and for $\mathcal{P}$ to prove the following relation holds:

$$\langle (1, \gamma, \gamma^2, \dots), v \rangle \in \left\{ \langle (1, \gamma, \gamma^2, \dots), tv^{(i)} \rangle \right\}_{i \in [B]}$$

This method is highly efficient in the IT-MAC setting, as all linear operations can be performed locally without additional communication. Essentially, the parties compute the hash of the committed vector using the universal hashing approach,

¹⁰ For example, the circuit may contain a MULT gate with a public constant as one of its inputs; the “1” is necessary to support such gates — see Figure 10 in Appendix A.3.

and $\mathcal{P}$ proves that this hash matches one of the B publicly known hash values corresponding to the topology vectors.

Similarly, IT-PACs enjoy free linear operations. Once $\mathcal{V}$ samples and sends the challenge γ , the parties can linearly combine the committed polynomials induced by active topology vectors (from Step 4). Crucially, for each $j \in [R]$, evaluating the aggregated polynomial at α_j yields the hash of the committed active topology vector for the j -th repetition!

However, this alone is insufficient, as we still need a mechanism to verify *in ZK* that the resulting hash (at each α) belongs to the public set of (topology) hashes. To address this, the key observation leveraged by **Justvengers** is that $\mathcal{P}$ can efficiently commit to this information using polynomials.

In particular, $\mathcal{P}$ can construct a $B \times R$ matrix MK as follows:

$$\forall i \in [B], j \in [R], MK_{i,j} = \begin{cases} 1, & \text{if the } i\text{-th branch is active in the } j\text{-th repetition} \\ 0, & \text{otherwise} \end{cases}$$

Namely, the matrix MK marks which branch is active in each repetition. $\mathcal{P}$ then applies interpolation row-by-row on MK and commits to the resulting polynomials using IT-PACs, *before γ is issued*, depicted as follows:

$$\begin{array}{ccc} MK & \xRightarrow{\text{Inter}} & \begin{array}{c} MK_1(\cdot) \\ \dots \\ MK_B(\cdot) \end{array} & \xRightarrow{\text{Commit}} & \begin{array}{c} [MK_1(\cdot)] \\ \dots \\ [MK_B(\cdot)] \end{array} \end{array}$$

Once $\mathcal{V}$ samples and sends γ , the parties compute B *public* hash values over the topology vectors as $h_{i \in [B]} := \langle (1, \gamma, \gamma^2, \dots), \mathbf{tv}^{(i)} \rangle$. It then suffices for $\mathcal{P}$ to prove that the following polynomial is vanishing:¹¹

$$\sum_{k=1}^{n_{in}+3n_{\times}+1} \gamma^{k-1} \cdot TV_k(\cdot) - \sum_{i=1}^B h_i \cdot MK_i(\cdot)$$

Thanks to the linear homomorphism of IT-PACs, the commitment to this polynomial can be computed locally. (Note, since its degree is strictly less than R , the polynomial must be identically zero if it vanishes at all $\alpha_{j \in [R]}$.)

Finally, $\mathcal{P}$ must prove that each entry in the matrix MK is either 0 or 1. This can be done straightforwardly: for each $i \in [B]$, the parties generate the IT-PAC for the polynomial $[MK_i(\cdot)(MK_i(\cdot) - 1)]$ and prove that it is vanishing. Moreover, parties need to verify that each column of MK contains exactly a single one, ensuring a valid branch selection per repetition. This could be efficiently checked by proving that $\sum_{i \in [B]} MK_i(\cdot)$ is the unit polynomial.

Cost: This step in **Justvengers** requires $\mathcal{O}(B)$ communication (plus a one-time $\mathcal{O}(R)$ cost for batching the vanishing polynomial proofs) and $\mathcal{O}(B)$ VOLE correlations. This matches that of **Batchman**.

¹¹ Essentially, we compute the same hash in two different ways.

Conclusion. In summary, **Justvengers** achieves $\mathcal{O}(R + B + C)$ communication and consumes $\mathcal{O}(R + B + C)$ VOLE correlations. Applying Lemma 2, the overall communication cost remains $\mathcal{O}(R + B + C)$ in the plain model (assuming LPN).

Important Note. For simplicity, several low-level but critical details have been omitted in this section. A full formalization with them is presented in Section 4. Here, we briefly highlight a few key aspects as a preview:

- The batched disjunctive setting is only meaningful when the witness can be “soldered” across repetitions, as supported by **Justvengers**; see Section 4.3.
- Generating IT-PACs requires the use of AHE. **Justvengers** relies solely on an AHE scheme without *verifiability*, made possible by the “commit-disclose-open” technique [JKO13, WYY⁺22]; see Section 4.1.
- Multiplication of two IT-PACs depends on the fact that Λ can be safely disclosed at some stage; see Section 4.1.
- We omit the computational complexity analysis in this section. As in **Antman**, the main bottleneck in **Justvengers** is NTT-based polynomial interpolation, and it requires $\mathcal{O}((B + C)R \log R)$ field operations; see Section 4.2.
- The protocol described in this section does not provide a proof of knowledge, i.e., the simulator *cannot* extract the inputs used by $\mathcal{P}$ in each repetition as they were committed inside polynomials. To address this, we follow the approach of **Antman**, where each input-related polynomial coefficient is additionally committed as an IT-MAC; see Section 4.1. Crucially, this modification does *not* affect overall communication asymptotic efficiency, since it adds only $\mathcal{O}(Rn_{in})$ overhead.

4 Formalization

4.1 UC Description of **Justvengers**

We formalize our protocol **Justvengers** as $\Pi_{\text{Justvengers}}$ in Figures 4 to 9. See Section 3 for a concise technical overview.

The formal definition of **Justvengers** consists of five major phases:

1. **Initialization** (Figure 4): In this phase, parties set up the protocol by generating (1) AHE ciphertexts over Λ and its powers, and (2) a pool of VOLE correlations to be used in subsequent phases. Notably, in Step 2, $\mathcal{V}$ commits to the seed used for generating AHE randomness. This commitment is critical for security, as a malicious $\mathcal{V}$ could otherwise send ill-formed AHE ciphertexts. Later in the protocol, $\mathcal{V}$ is required to reveal this seed, allowing $\mathcal{P}$ to verify the correctness of the ciphertexts.
2. **Commitment** (Figures 5 and 6): In this phase, $\mathcal{P}$ commits to the computed AHE ciphertexts (used to generate IT-PACs) via $\mathcal{F}_{\text{Com}}$. This step is necessary because $\mathcal{P}$ has not yet verified the well-formedness of the ciphertexts from $\mathcal{V}$, and revealing them prematurely could compromise ZK. Note, in Step 9, $\mathcal{P}$ additionally commits — using IT-MACs — to the coefficients of the input polynomials $IN_{k \in [n_{in}]}(\cdot)$. This is required for simulation,

as the witness must be extractable, which is not possible from unopened polynomials alone. These extra commitments do not affect asymptotic efficiency, as it applies only to the input layer. Moreover, these IT-MACs are unrelated to the AHE ciphertexts and thus do not require the usage of $\mathcal{F}_{\text{Com}}$. During execution, $\mathcal{P}$ also maintains an ordered queue Q_{vanish} (initially empty) to track vanishing polynomials that need to be proved.

Finally, we note that the six steps in this phase correspond directly to those described in Section 3.3.

3. **Disclosure** (Figure 7): In this phase, $\mathcal{P}$ proves in ZK that all polynomials in Q_{vanish} are vanishing. To do so, in Step 11, $\mathcal{P}$ samples a random vanishing polynomial $PAD(\cdot)$ and commits to it using IT-PAC (again, this goes to $\mathcal{F}_{\text{Com}}$). In Step 12, $\mathcal{V}$ sends a random challenge ζ , which is used to linearly combine all vanishing polynomials. The resulting aggregated polynomial is one-time padded with $PAD(\cdot)$, forming $VP(\cdot)$, which $\mathcal{P}$ reveals to $\mathcal{V}$.

Crucially, since $VP(\cdot)$ is independent of AHE, it does not require a commit-then-reveal strategy. However, it must be sent before $\mathcal{P}$ learns Δ — otherwise, a malicious $\mathcal{P}$ can always construct a vanishing polynomial $\widetilde{VP}(\cdot)$ that evaluates to an arbitrary value at Δ , violating soundness.

In the final Step 13 of this phase, $\mathcal{V}$ reveals the committed seed and the value of Δ used to generate the corresponding AHE ciphertexts. This allows $\mathcal{P}$ to verify their correctness. If the ciphertexts are ill-formed, $\mathcal{P}$ aborts. Note that up to this point, $\mathcal{V}$ has learned only uniformly random values besides some handles from $\mathcal{F}_{\text{Com}}$, so this abort preserves ZK. If the ciphertexts are well-formed, then other AHE ciphertexts generated by $\mathcal{P}$ are simply encryptions of one-time-padded values, which can now be safely revealed to $\mathcal{V}$.

4. **Open** (Figure 8): In this phase, $\mathcal{P}$ reveals all previously committed AHE ciphertexts (in $\mathcal{F}_{\text{Com}}$) to $\mathcal{V}$. Since $\mathcal{P}$ now knows Δ , all IT-PACs become IT-MACs.¹² That is, $\mathcal{P}$ can evaluate each committed polynomial at Δ . Crucially, knowing Δ does *not* enable $\mathcal{P}$ to forge any committed values within the IT-MACs, as the global key Δ still retains full entropy.
5. **Verification** (Figure 9): In this phase, parties perform several remaining consistency checks. Step 20 verifies that the revealed (random-looking) vanishing polynomial is the correct one intended to be opened. This is done by checking whether $VP(\Delta)$ matches the value committed in the corresponding IT-MAC. The check is sound as $\mathcal{P}$ opens the polynomial before learning Δ — any deviation would match at Δ with probability $\mathcal{O}\left(\frac{R}{|\mathbb{F}|}\right)$ by Lemma 1.

Step 21 ensures that the committed polynomial correctly represents the product of two input polynomials. This follows the same technique used in Antman [WYY⁺22].

Step 22 verifies that the IT-MACs of the committed coefficients match those used in the corresponding interpolated polynomials. This step ensures that the extracted witness aligns with what $\mathcal{P}$ actually uses in the proof (encoded inside committed polynomial IT-PACs).

¹² To some extent, IT-PACs are used purely as a conceptual tool in the formalization.

Protocol $\Pi_{\text{Justvengers}}$, Part I

Let $\text{AHE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ be an additively homomorphic encryption scheme (defined in Appendix A.1). Let $G : \{0, 1\}^\lambda \mapsto \{0, 1\}^*$ be a PRG.

Inputs. $\mathcal{P}$ and $\mathcal{V}$ hold B circuits $\mathcal{C}_1, \dots, \mathcal{C}_B$ over field $\mathbb{F}$, where each circuit has n_{in} inputs, $n_\times$ multiplications and 1 output. $\mathcal{P}$ also holds R witness $\mathbf{w}_j \in \mathbb{F}^{n_{in}}$ and R integers $id_j \in [B]$ for each $j \in [R]$ such that $\forall j \in [R], \mathcal{C}_{id_j}(\mathbf{w}_j) = 0$. Suppose that $\mathcal{P}$ and $\mathcal{V}$ have already agreed on (1) a set of public parameters $\text{par} = \text{Setup}(1^\lambda)$ for AHE and (2) R distinct fixed points $\alpha_1, \dots, \alpha_R \in \mathbb{F}$.

Initialization.

1. $\mathcal{P}$ and $\mathcal{V}$ send **(init)** to $\mathcal{F}_{\text{Vole}}$, which returns a uniform $\Delta \xleftarrow{\$} \mathbb{F}$ to $\mathcal{V}$.
2. $\mathcal{V}$ samples a uniform $\text{seed} \xleftarrow{\$} \{0, 1\}^\lambda$, and sends **(commit, seed, τ_{seed})** to $\mathcal{F}_{\text{Com}}$, which returns the handle τ_{seed} to $\mathcal{P}$. $\mathcal{P}$ aborts if she does not receive it.
3. $\mathcal{V}$ samples a uniform $A \xleftarrow{\$} \mathbb{F}$ and expands the PRG $G(\text{seed})$ into a bit string $(r_0, r_1, \dots, r_{2R-2})$, where r_0 is used for **KeyGen** and each r_j (for $j \in [2R-2]$) is used for **Enc**. $\mathcal{V}$ then computes the secret key $\text{sk} := \text{KeyGen}(\text{par}; r_0)$ and encrypts A^j as $\llbracket A^j \rrbracket := \text{Enc}(\text{sk}, A^j; r_j)$ for each $j \in [2R-2]$. Finally, $\mathcal{V}$ sends $\llbracket A \rrbracket, \dots, \llbracket A^{2R-2} \rrbracket$ to $\mathcal{P}$.
4. $\mathcal{P}$ and $\mathcal{V}$ send **(extend, m)** to $\mathcal{F}_{\text{Vole}}$, which returns a pool of m VOLE correlations (i.e., random IT-MACs), where $m = Rn_{in} + 2B + 3n_{in} + 10n_\times + 3 \in \mathcal{O}(R+B+C)$. Throughout the remainder of the protocol, any reference to a random IT-MAC or a single VOLE correlation refers to retrieving a new entry from this pre-allocated pool.^a

^a We assume that $\mathcal{P}$ and $\mathcal{V}$ retrieve them in a consistent, predetermined order.

Fig. 4: Justvengers (Part I): ZKP protocol for the batched disjunctive statement over $\mathbb{F}$ in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{Vole}}\}$ -hybrid (Figures 2 and 3) model.

Protocol $\Pi_{\text{Justvengers}}$, Part II

Commitment.

5. For the j -th repetition ($j \in [R]$), $\mathcal{P}$ evaluates $\mathcal{C}_{id_j}(\mathbf{w}_j)$ and records corresponding extended witness including $in_{k \in [n_{in}]}^{(j)}$ for circuit inputs, $\ell_{k \in [n_{\times}]}^{(j)}$ (resp. $r_{k \in [n_{\times}]}^{(j)}$ and $o_{k \in [n_{\times}]}^{(j)}$) for left inputs (resp. right inputs and outputs) of MULT gates. Then, for each $k \in [n_{in}]$, $\mathcal{P}$ applies the interpolation procedure to obtain the polynomial $IN_k(\cdot) := \text{Inter}(in_k^{(1)}, \dots, in_k^{(R)})$; and $\mathcal{P}$ computes $\llbracket IN_k(\Lambda) - s_k^{(in)} \rrbracket$ by using additive homomorphism based on $\llbracket \Lambda \rrbracket, \dots, \llbracket \Lambda^{R-1} \rrbracket$ and a VOLE correlation $\left[s_k^{(in)} \right]$. The same applies to $L_k(\cdot), R_k(\cdot), O_k(\cdot)$ and corresponding $\llbracket L_k(\Lambda) - s_k^{(\ell)} \rrbracket, \llbracket R_k(\Lambda) - s_k^{(r)} \rrbracket, \llbracket O_k(\Lambda) - s_k^{(o)} \rrbracket$ for each $k \in [n_{\times}]$. $\mathcal{P}$ commits to the above $n_{in} + 3n_{\times}$ AHE ciphertexts via $\mathcal{F}_{\text{Com}}$ using the handle τ_{wit} , which is then sent to $\mathcal{V}$. $\mathcal{V}$ aborts if he does not receive a handle. Finally, for each $k \in [n_{in}]$, let R coefficients on $IN_k(\cdot)$ be $c_{k,0}^{(in)}, \dots, c_{k,R-1}^{(in)}$: $\mathcal{P}$ and $\mathcal{V}$ generate R IT-MACs $\left[c_{k,0}^{(in)} \right], \dots, \left[c_{k,R-1}^{(in)} \right]$ from R VOLE correlations. $\mathcal{V}$ aborts if he fails to build these IT-MACs.
6. For each $k \in [n_{\times}]$, $\mathcal{P}$ computes $L_k(\cdot)R_k(\cdot)$ and $\llbracket L_k(\Lambda)R_k(\Lambda) - s_k^{(\ell r)} \rrbracket$ by using additive homomorphism based on $\llbracket \Lambda \rrbracket, \dots, \llbracket \Lambda^{2R-2} \rrbracket$ and a VOLE correlation $\left[s_k^{(\ell r)} \right]$; and $\mathcal{P}$ appends the polynomial $L_k(\cdot)R_k(\cdot) - O_k(\cdot)$ into the queue Q_{vanish} . $\mathcal{P}$ commits to the above $n_{\times}$ AHE ciphertexts via $\mathcal{F}_{\text{Com}}$ using the handle τ_{mul} , which is then sent to $\mathcal{V}$. $\mathcal{V}$ aborts if he does not receive a handle.
7. $\mathcal{V}$ issues a random challenge χ . Parties compute the length- $(n_{in} + 3n_{\times} + 1)$ topology vector $\mathbf{tv}^{(i)}$ for each $\mathcal{C}_{i \in [B]}$; see [YHH⁺23a, Section 4].
8. $\mathcal{P}$ constructs the $(n_{in} + 3n_{\times} + 1) \times R$ matrix $TV := (\mathbf{tv}^{(id_1)}, \dots, \mathbf{tv}^{(id_R)})$. Then $\mathcal{P}$ applies the interpolation procedure over TV (see Section 3.2) to get polynomials $TV_1(\cdot), \dots, TV_{n_{in} + 3n_{\times} + 1}(\cdot)$. For each $k \in [n_{in} + 3n_{\times} + 1]$, $\mathcal{P}$ computes $\llbracket TV_k(\Lambda) - s_k^{(tv)} \rrbracket$ by using additive homomorphism based on $\llbracket \Lambda \rrbracket, \dots, \llbracket \Lambda^{R-1} \rrbracket$ and a VOLE correlation $\left[s_k^{(tv)} \right]$. $\mathcal{P}$ commits to the above $n_{in} + 3n_{\times} + 1$ AHE ciphertexts via $\mathcal{F}_{\text{Com}}$ using the handle τ_{tv} , which is then sent to $\mathcal{V}$. $\mathcal{V}$ aborts if he does not receive a handle.

Fig. 5: Justvengers (Part II): ZKP protocol for the batched disjunctive statement over $\mathbb{F}$ in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{VOLE}}\}$ -hybrid (Figures 2 and 3) model.

Protocol $\Pi_{\text{Justvengers}}$, Part III

Commitment.

9. $\mathcal{P}$ computes the following:

- (a) For each $k \in [n_{in}]$: $IN_k(\cdot)TV_k(\cdot)$ and $\llbracket IN_k(\Lambda)TV_k(\Lambda) - s_k^{(intv)} \rrbracket$ by using additive homomorphism and a VOLE correlation $[s_k^{(intv)}]$.
- (b) For each $k \in [n_{\times}]$:
 $L_k(\cdot)TV_{n_{in}+k}(\cdot)$ and $\llbracket L_k(\Lambda)TV_{n_{in}+k}(\Lambda) - s_k^{(ltv)} \rrbracket$ by using additive homomorphism and a VOLE correlation $[s_k^{(ltv)}]$.
 $R_k(\cdot)TV_{n_{in}+n_{\times}+k}(\cdot)$ and $\llbracket R_k(\Lambda)TV_{n_{in}+n_{\times}+k}(\Lambda) - s_k^{(rtv)} \rrbracket$ by using additive homomorphism and a VOLE correlation $[s_k^{(rtv)}]$.
 $O_k(\cdot)TV_{n_{in}+2n_{\times}+k}(\cdot)$ and $\llbracket O_k(\Lambda)TV_{n_{in}+2n_{\times}+k}(\Lambda) - s_k^{(otv)} \rrbracket$ by using additive homomorphism and a VOLE correlation $[s_k^{(otv)}]$.
- (c) The polynomial $IP(\cdot) =$

$$\begin{aligned} \sum_{k=1}^{n_{in}} IN_k(\cdot)TV_k(\cdot) + \sum_{k=1}^{n_{\times}} L_k(\cdot)TV_{n_{in}+k}(\cdot) + \sum_{k=1}^{n_{\times}} R_k(\cdot)TV_{n_{in}+n_{\times}+k}(\cdot) \\ + \sum_{k=1}^{n_{\times}} O_k(\cdot)TV_{n_{in}+2n_{\times}+k}(\cdot) + TV_{n_{in}+3n_{\times}+1}(\cdot) \end{aligned}$$

$\mathcal{P}$ commits to the above $n_{in}+3n_{\times}$ AHE ciphertexts via $\mathcal{F}_{\text{Com}}$ using the handle τ_p , which is then sent to $\mathcal{V}$. $\mathcal{V}$ aborts if he does not receive a handle. $\mathcal{P}$ appends $IP(\cdot)$ into the queue Q_{vanish} .

- 10. $\mathcal{P}$ constructs a $B \times R$ matrix MK such that for all $i \in [B]$ and $j \in [R]$, $MK_{i,j} = 1$ if $i = id_j$, and 0 otherwise. $\mathcal{P}$ applies the interpolation procedure over MK (see Section 3.2) to get polynomials $MK_1(\cdot), \dots, MK_B(\cdot)$. For each $i \in [B]$, $\mathcal{P}$ computes $\llbracket MK_i(\Lambda) - s_i^{(mk)} \rrbracket$ by using AH and a VOLE correlation $[s_i^{(mk)}]$.
 $\mathcal{P}$ computes $MK_i(\cdot)(MK_i(\cdot) - 1)$ and $\llbracket MK_i(\Lambda)(MK_i(\Lambda) - 1) - s_i^{(one)} \rrbracket$, for each $i \in [B]$, by using additive homomorphism and a VOLE correlation $[s_i^{(one)}]$; and $\mathcal{P}$ appends the polynomial $MK_i(\cdot)(MK_i(\cdot) - 1)$ into the queue Q_{vanish} .
 $\mathcal{P}$ commits to the above $2B$ AHE ciphertexts via $\mathcal{F}_{\text{Com}}$ using the handle τ_{mk} , which is then sent to $\mathcal{V}$. $\mathcal{V}$ aborts if he does not receive a handle. $\mathcal{V}$ issues a random challenge $\gamma \xleftarrow{\$} \mathbb{F}$, and $\mathcal{P}$ appends the following two polynomials into the queue Q_{vanish} : (1) $\sum_{i=1}^B MK_i(\cdot) - 1$ and (2) $\sum_{k=1}^{n_{in}+3n_{\times}+1} \gamma^{k-1} \cdot TV_k(\cdot) - \sum_{i=1}^B h_i \cdot MK_i(\cdot)$ where $h_{i \in [B]} := \langle (1, \gamma, \gamma^2, \dots), tv^{(i)} \rangle$.

Fig. 6: Justvengers (Part III): ZKP protocol for the batched disjunctive statement over $\mathbb{F}$ in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{VOLE}}\}$ -hybrid (Figures 2 and 3) model.

Protocol $\Pi_{\text{Justvengers}}$, Part IV

Disclosure.

11. $\mathcal{P}$ samples a random vanishing polynomial $PAD(\cdot)$ of degree $2R-2$. Then $\mathcal{P}$ computes $\llbracket PAD(\Lambda) - s^{(pad)} \rrbracket$ by using additive homomorphism based on $\llbracket \Lambda \rrbracket, \dots, \llbracket \Lambda^{2R-2} \rrbracket$ and a VOLE correlation $\left[s^{(pad)} \right]$. $\mathcal{P}$ commits to the above AHE ciphertext via $\mathcal{F}_{\text{Com}}$ using the handle τ_{pad} , which is then sent to $\mathcal{V}$. $\mathcal{V}$ aborts if he does not receive a handle.
12. $\mathcal{V}$ samples a random element $\zeta \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$. Then $\mathcal{P}$ computes and sends the following vanishing polynomial (i.e., its coefficients):

$$VP(\cdot) := PAD(\cdot) + \sum_{f_i(\cdot) \in Q_{\text{vanish}} \text{ in order}} \zeta^i f_i(\cdot)$$

Upon receiving $VP(\cdot)$, $\mathcal{V}$ checks if it is vanishing; if not, he aborts.

13. $\mathcal{V}$ sends $(\text{open}, \tau_{\text{seed}})$ to $\mathcal{F}_{\text{Com}}$, which returns $(\text{seed}, \tau_{\text{seed}})$ to $\mathcal{P}$. In parallel, $\mathcal{V}$ sends Λ to $\mathcal{P}$. If $\mathcal{P}$ does not receive them, she aborts; otherwise, $\mathcal{P}$ expands the PRG $G(\text{seed})$ into a bit string $(r_0, r_1, \dots, r_{2R-2})$, where r_0 is used for KeyGen and each r_j (for $j \in [2R-2]$) is used for Enc . $\mathcal{P}$ then computes the secret key $\text{sk} := \text{KeyGen}(\text{par}; r_0)$ and checks whether $\forall j \in [2R-2], \llbracket \Lambda^j \rrbracket \stackrel{?}{=} \text{Enc}(\text{sk}, \Lambda^j; r_j)$ (i.e., those AHE ciphertexts sent by $\mathcal{V}$ in Step 3); if not, she aborts.

Fig. 7: Justvengers (Part IV): ZKP protocol for the batched disjunctive statement over $\mathbb{F}$ in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{VOLE}}\}$ -hybrid (Figures 2 and 3) model.

Protocol $\Pi_{\text{Justvengers}}$, Part V

Open.

14. $\mathcal{P}$ sends $(\text{open}, \tau_{\text{wit}})$ to $\mathcal{F}_{\text{Com}}$, which returns the $n_{\text{in}} + 3n_{\times}$ AHE ciphertexts $\mathcal{P}$ committed to in Step 5. If $\mathcal{V}$ does not receive them, he aborts. Then $\mathcal{V}$ decrypts these ciphertexts. Since $\mathcal{P}$ knows Λ , parties can obtain the following IT-MACs:

$$[IN_1(\Lambda)], \dots, [IN_{n_{\text{in}}}(\Lambda)], \\ [L_1(\Lambda)], \dots, [L_{n_{\times}}(\Lambda)], [R_1(\Lambda)], \dots, [R_{n_{\times}}(\Lambda)], [O_1(\Lambda)], \dots, [O_{n_{\times}}(\Lambda)]$$

15. $\mathcal{P}$ sends $(\text{open}, \tau_{\text{mul}})$ to $\mathcal{F}_{\text{Com}}$, which returns the $n_{\times}$ AHE ciphertexts $\mathcal{P}$ committed to in Step 6. If $\mathcal{V}$ does not receive them, he aborts. Then $\mathcal{V}$ decrypts these ciphertexts. Since $\mathcal{P}$ knows Λ , parties can obtain the following IT-MACs:

$$[L_1(\Lambda)R_1(\Lambda)], \dots, [L_{n_{\times}}(\Lambda)R_{n_{\times}}(\Lambda)]$$

16. $\mathcal{P}$ sends $(\text{open}, \tau_{\text{tv}})$ to $\mathcal{F}_{\text{Com}}$, which returns the $n_{\text{in}} + 3n_{\times} + 1$ AHE ciphertexts $\mathcal{P}$ committed to in Step 8. If $\mathcal{V}$ does not receive them, he aborts. Then $\mathcal{V}$ decrypts these ciphertexts. Since $\mathcal{P}$ knows Λ , parties can obtain the following IT-MACs:

$$[TV_1(\Lambda)], \dots, [TV_{n_{\text{in}} + 3n_{\times} + 1}(\Lambda)]$$

17. $\mathcal{P}$ sends $(\text{open}, \tau_{\text{ip}})$ to $\mathcal{F}_{\text{Com}}$, which returns the $n_{\text{in}} + 3n_{\times}$ AHE ciphertexts $\mathcal{P}$ committed to in Step 9. If $\mathcal{V}$ does not receive them, he aborts. Then $\mathcal{V}$ decrypts these ciphertexts. Since $\mathcal{P}$ knows Λ , parties can obtain the following IT-MACs:

$$[IN_1(\Lambda)TV_1(\Lambda)], \dots, [IN_{n_{\text{in}}}(\Lambda)TV_{n_{\text{in}}}(\Lambda)] \\ [L_1(\Lambda)TV_{n_{\text{in}}+1}(\Lambda)], \dots, [L_{n_{\times}}(\Lambda)TV_{n_{\text{in}}+n_{\times}}(\Lambda)] \\ [R_1(\Lambda)TV_{n_{\text{in}}+n_{\times}+1}(\Lambda)], \dots, [R_{n_{\times}}(\Lambda)TV_{n_{\text{in}}+2n_{\times}}(\Lambda)] \\ [O_1(\Lambda)TV_{n_{\text{in}}+2n_{\times}+1}(\Lambda)], \dots, [O_{n_{\times}}(\Lambda)TV_{n_{\text{in}}+3n_{\times}}(\Lambda)]$$

18. $\mathcal{P}$ sends $(\text{open}, \tau_{\text{mk}})$ to $\mathcal{F}_{\text{Com}}$, which returns the $2B$ AHE ciphertexts $\mathcal{P}$ committed to in Step 10. If $\mathcal{V}$ does not receive them, he aborts. Then $\mathcal{V}$ decrypts these ciphertexts. Since $\mathcal{P}$ knows Λ , parties can obtain the following IT-MACs:

$$[MK_1(\Lambda)], \dots, [MK_B(\Lambda)], [MK_1(\Lambda)(MK_1(\Lambda) - 1)], \dots, [MK_B(\Lambda)(MK_B(\Lambda) - 1)]$$

19. $\mathcal{P}$ sends $(\text{open}, \tau_{\text{pad}})$ to $\mathcal{F}_{\text{Com}}$, which returns one AHE ciphertext $\mathcal{P}$ committed to in Step 11. If $\mathcal{V}$ does not receive them, he aborts. Then $\mathcal{V}$ decrypts these ciphertexts. Since $\mathcal{P}$ knows Λ , parties can obtain the IT-MAC $[PAD(\Lambda)]$.

Fig. 8: Justvengers (Part V): ZKP protocol for the batched disjunctive statement over $\mathbb{F}$ in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{VOLE}}\}$ -hybrid (Figures 2 and 3) model.

Protocol $\Pi_{\text{Justvengers}}$, Part VI

Verification.

20. **Parties verify the opened vanishing polynomial.** Parties *locally* compute the following IT-MAC:

$$\begin{aligned}
& [PAD(\Lambda)] \\
& + \sum_{k=1}^{n_{\times}} \zeta^k ([L_k(\Lambda) R_k(\Lambda)] - [O_k(\Lambda)]) + \sum_{k=1}^{n_{in}} \zeta^{n_{\times}+1} [IN_k(\Lambda) TV_k(\Lambda)] \\
& + \sum_{k=1}^{n_{\times}} \zeta^{n_{\times}+1} [L_k(\Lambda) TV_{n_{in}+k}(\Lambda)] + \sum_{k=1}^{n_{\times}} \zeta^{n_{\times}+1} [R_k(\Lambda) TV_{n_{in}+n_{\times}+k}(\Lambda)] \\
& + \sum_{k=1}^{n_{\times}} \zeta^{n_{\times}+1} [O_k(\Lambda) TV_{n_{in}+2n_{\times}+k}(\Lambda)] + \zeta^{n_{\times}+1} [TV_{n_{in}+3n_{\times}+1}(\Lambda)] \\
& + \sum_{i=1}^B \zeta^{n_{\times}+1+i} [MK_i(\Lambda)(MK_i(\Lambda) - 1)] \\
& + \zeta^{n_{\times}+B+2} \left(\sum_{i=1}^B [MK_i(\Lambda)] - 1 \right) \\
& + \zeta^{n_{\times}+B+3} \left(\sum_{k=1}^{n_{in}+3n_{\times}+1} \gamma^{k-1} [TV_k(\Lambda)] - \sum_{i=1}^B h_i [MK_i(\Lambda)] \right)
\end{aligned}$$

Then $\mathcal{P}$ opens the committed value to $\mathcal{V}$. If the value is not $VP(\Lambda)$, $\mathcal{V}$ aborts.

21. **Parties verify polynomial multiplications.** Parties execute LPZK (see Section 2.5) to verify the following multiplication of IT-MAC triples:

$$\begin{aligned}
& \{[L_k(\Lambda)], [R_k(\Lambda)], [L_k(\Lambda) R_k(\Lambda)]\}_{k \in [n_{\times}]} \\
& \{[IN_k(\Lambda)], [TV_k(\Lambda)], [IN_k(\Lambda) TV_k(\Lambda)]\}_{k \in [n_{in}]} \\
& \{[L_k(\Lambda)], [TV_{n_{in}+k}(\Lambda)], [L_k(\Lambda) TV_{n_{in}+k}(\Lambda)]\}_{k \in [n_{\times}]} \\
& \{[R_k(\Lambda)], [TV_{n_{in}+n_{\times}+k}(\Lambda)], [R_k(\Lambda) TV_{n_{in}+n_{\times}+k}(\Lambda)]\}_{k \in [n_{\times}]} \\
& \{[O_k(\Lambda)], [TV_{n_{in}+2n_{\times}+k}(\Lambda)], [O_k(\Lambda) TV_{n_{in}+2n_{\times}+k}(\Lambda)]\}_{k \in [n_{\times}]} \\
& \{[MK_i(\Lambda)], [MK_i(\Lambda) - 1], [MK_i(\Lambda)(MK_i(\Lambda) - 1)]\}_{i \in [B]}
\end{aligned}$$

22. **Parties verify committed inputs.** For each $k \in [n_{in}]$, $\mathcal{P}$ proves to $\mathcal{V}$ that $\left[c_{k,0}^{(in)} \right] + \sum_{j=1}^{R-1} \Lambda^j \left[c_{k,j}^{(in)} \right] - [IN_k(\Lambda)]$ commits to a zero.

Output. If $\mathcal{V}$ aborted earlier, he outputs (**false**); otherwise, he outputs (**true**).

Fig. 9: Justvengers (Part VI): ZKP protocol for the batched disjunctive statement over $\mathbb{F}$ in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{Vole}}\}$ -hybrid (Figures 2 and 3) model.

4.2 Analysis and Variants

In this section, we present the main theorem and its simulation-based proof. We also analyze $\Pi_{\text{Justvengers}}$'s efficiency and discuss several variants that trade off between (concrete) efficiency, underlying assumptions, and soundness error.

Theorem 1 (main). *If the AHE scheme satisfies the CPA security, circuit privacy, and linear targeted malleability along with G is a pseudorandom generator, $\Pi_{\text{Justvengers}}$ (Figures 4 to 9) UC-realizes $\mathcal{F}_{\text{ZK}}^{R,B}$ (Figure 1) in the $\{\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{VOLE}}\}$ -hybrid model (Figures 2 and 3) in the presence of a static adversary.*

Proof. The proof is performed by constructing the simulator $\mathcal{S}$. We need to show **completeness** (trivial, omitted); **soundness** (constructing $\mathcal{S}$ for $\mathcal{P}^*$); and **ZK** (constructing $\mathcal{S}$ for $\mathcal{V}^*$).

ZK, $\mathcal{S}$ for $\mathcal{V}^*$: We start by constructing the $\mathcal{S}$ for $\mathcal{V}^*$ since it is relatively straightforward. At a high level, all messages received by $\mathcal{V}$ (excluding handles from $\mathcal{F}_{\text{Com}}$) fall into one of two categories: (1) uniformly random field elements (or a uniformly random vanishing polynomial), or (2) proofs that depend solely on values locally known to $\mathcal{V}$ — i.e., any valid proof $\mathcal{V}$ expects from $\mathcal{P}$ corresponds to field elements $\mathcal{V}$ can already compute. As a result, the main challenge for constructing the ZK simulator lies in generating these proof values. However, this is straightforward because $\mathcal{S}$ emulates the $\mathcal{F}_{\text{VOLE}}$ hybrid for $\mathcal{V}$ and thus has access to $\mathcal{V}$'s full view, including the global IT-MAC key Δ and all local IT-MAC keys. In detail, $\mathcal{S}$ emulates $\mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{Com}}$ for $\mathcal{V}^*$ and interacts with $\mathcal{V}^*$ as follows:

- For Step 1, $\mathcal{S}$ samples the Δ for $\mathcal{V}^*$. Note that $\mathcal{V}^*$ can specify its own Δ by revealing its Δ to $\mathcal{S}$ (i.e., to the hybrid functionality $\mathcal{F}_{\text{VOLE}}$).
- For Step 2, $\mathcal{S}$ receives seed from $\mathcal{V}^*$.
- For Step 3, $\mathcal{S}$ receives $2R-2$ AHE ciphertexts from $\mathcal{V}^*$. Note that $\mathcal{V}^*$ may send ill-formed ciphertexts — we will catch this later.
- For Step 4, $\mathcal{S}$ samples the local keys (i.e., the $\mathcal{V}^*$'s IT-MAC shares of VOLE correlations) for $\mathcal{V}^*$. Note that $\mathcal{V}^*$ can specify its own local keys by revealing its local keys to $\mathcal{S}$ (i.e., to the hybrid functionality $\mathcal{F}_{\text{VOLE}}$).
- For Step 5, $\mathcal{S}$ sends the handle τ_{wit} to $\mathcal{V}^*$, and Rn_{in} random field elements to $\mathcal{V}^*$. This perfectly mirrors the behavior of the real-world $\mathcal{P}$, who sends values that are one-time padded with random values inside VOLE correlations.
- For Step 6, $\mathcal{S}$ sends the handle τ_{mul} to $\mathcal{V}^*$.
- For Step 7, $\mathcal{S}$ receives χ from $\mathcal{V}^*$.
- For Step 8, $\mathcal{S}$ sends the handle τ_{tv} to $\mathcal{V}^*$.
- For Step 9, $\mathcal{S}$ sends the handle τ_{ip} to $\mathcal{V}^*$.
- For Step 10, $\mathcal{S}$ sends the handle τ_{mk} to $\mathcal{V}^*$.
- For Step 11, $\mathcal{S}$ sends the handle τ_{pad} to $\mathcal{V}^*$.
- For Step 12, $\mathcal{S}$ receives ζ from $\mathcal{V}^*$ and sends a random vanishing polynomial of degree $2R-2$ to $\mathcal{V}^*$.
- For Step 13, $\mathcal{S}$ receives Λ from $\mathcal{V}^*$ and expands the PRG $G(\text{seed})$ to evaluate AHE.KeyGen and AHE.Enc to get the secret key sk as well as check if all AHE ciphertexts (over Λ and its powers) sent by $\mathcal{V}^*$ are well-formed. If not, $\mathcal{S}$ aborts. Note that this behaves exactly the same as the real-world $\mathcal{P}$.

- For Steps 14 to 19, $\mathcal{S}$ emulates $\mathcal{F}_{\text{Com}}$ to open committed AHE ciphertexts. Specifically, for each ciphertext, $\mathcal{S}$ samples a random field element and encrypts it using sk . This mirrors the behavior of the real-world $\mathcal{P}$, who sends encryptions of values that are one-time padded with random values inside VOLE correlations. The only difference lies in how the encrypted uniform field elements are generated, which can be reduced to the circuit privacy property of AHE. (This is precisely why this property is required.)
- For Step 20, $\mathcal{S}$ performs the corresponding IT-MAC-related computation over $\mathcal{V}^*$'s local keys, denoted p . $\mathcal{S}$ sends $p + VP(\Delta)\Delta$ to $\mathcal{V}^*$.
- For Step 21, $\mathcal{S}$ calls the ZK simulator of LPZK — essentially, this is also done by emulating $\mathcal{V}^*$'s IT-MAC-related computation and then generating the proof based on Δ .
- For Step 22, $\mathcal{S}$ can simply emulate $\mathcal{V}^*$'s IT-MAC-related computation and then generate the proof based on Δ (basically, open the IT-MAC to a zero).

The distributions seen by $\mathcal{V}^*$ (and the UC environment) in the ideal world and the real world are *computationally indistinguishable*. Thus, ZK holds.

Soundness, $\mathcal{S}$ for $\mathcal{P}^*$: We now construct $\mathcal{S}$ for $\mathcal{P}^*$, beginning with a preview. In $\Pi_{\text{JustVengers}}$, the verifier $\mathcal{V}$ sends to the prover only the following: (1) uniform field elements, (2) a uniformly sampled seed, (3) handles via $\mathcal{F}_{\text{Com}}$, and (4) AHE ciphertexts over uniformly chosen Δ and its powers, with randomness derived from a PRG. Therefore, by emulating $\mathcal{F}_{\text{VOLE}}$ and $\mathcal{F}_{\text{Com}}$, $\mathcal{S}$ can interact with $\mathcal{P}^*$ exactly as an honest verifier would in the real protocol (i.e., $\mathcal{S}$ emulates $\mathcal{V}$).

Crucially, $\mathcal{S}$ is able to extract $\mathcal{P}^*$'s witness, as it is committed (in the form of polynomial coefficients) using IT-MACs in Step 5. Since $\mathcal{S}$ emulates $\mathcal{F}_{\text{VOLE}}$, it knows all one-time pads (from $\mathcal{P}^*$'s VOLE correlations) and can trivially remove them to recover the committed values.

If the emulated $\mathcal{V}$ aborts at any point, $\mathcal{S}$ sends **abort** to the ideal functionality $\mathcal{F}_{\text{ZK}}^{R,B}$, causing the ideal-world $\mathcal{V}$ to output **false**. Otherwise, $\mathcal{S}$ sends the extracted witness to $\mathcal{F}_{\text{ZK}}^{R,B}$.

Clearly, the views of $\mathcal{P}^*$ in the real and ideal worlds are identical. Furthermore, if the emulated $\mathcal{V}$ outputs **false**, then both the real-world and ideal-world $\mathcal{V}$ will also output **false**. Therefore, the only case that requires analysis is when the emulated $\mathcal{V}$ outputs **true** (i.e., the real-world $\mathcal{V}$ accepts), but the extracted witness is invalid — causing the ideal-world $\mathcal{V}$ to reject.

Namely, the core quantity to analyze is the probability that the emulated $\mathcal{V}$ outputs **true** while the extracted witness is incorrect. This corresponds to the soundness error, which we must show is negligible. As we will see, this probability can be bounded by carefully combining the soundness analyses from Antman [WYY⁺22] and Batchman [YHH⁺23a].

To give a complete view, we now present the full construction of $\mathcal{S}$, even though it is relatively straightforward (in contrast to the more involved soundness analysis). $\mathcal{S}$ emulates $\mathcal{F}_{\text{VOLE}}, \mathcal{F}_{\text{Com}}$ for $\mathcal{P}^*$ and interacts with $\mathcal{P}^*$ as follows:

- In Step 1, $\mathcal{S}$ receives **init** from $\mathcal{P}^*$ and also samples a uniform $\Delta \xleftarrow{\$} \mathbb{F}$.
- In Step 2, $\mathcal{S}$ samples a uniform seed $\xleftarrow{\$} \{0, 1\}^\lambda$ and sends τ_{seed} to $\mathcal{P}^*$.

- In Step 3, $\mathcal{S}$ samples a uniform $\Lambda \xleftarrow{\$} \mathbb{F}$ and expands the PRG $G(\text{seed})$. It then emulates $\mathcal{V}$ by generating AHE ciphertexts of Λ and its powers using randomness derived from the PRG, and sends these ciphertexts to $\mathcal{P}^*$.
- In Step 4, $\mathcal{S}$ samples the $\mathcal{P}^*$'s IT-MAC shares of VOLE correlations and sends them to $\mathcal{P}^*$. Note that $\mathcal{P}^*$ can specify its own shares by revealing them to $\mathcal{S}$ (i.e., to the hybrid functionality $\mathcal{F}_{\text{VOLE}}$). Now, $\mathcal{S}$ can compute emulated $\mathcal{V}$'s IT-MAC shares because it also has Δ .
- In Step 5, $\mathcal{S}$ receives $n_{in} + 3n_{\times}$ AHE ciphertexts (and τ_{wit}) from $\mathcal{P}^*$. It then receives Rn_{in} field elements used to commit — inside IT-MACs — to the coefficients of $IN_{k \in [n_{in}]}(\cdot)$. Since $\mathcal{S}$ emulates $\mathcal{F}_{\text{VOLE}}$, it can easily extract these coefficients. By evaluating the corresponding polynomials at the points α , $\mathcal{S}$ reconstructs the input witness for each repetition used by $\mathcal{P}^*$, denoted $\widetilde{\mathbf{w}}_{j \in [R]} \in \mathbb{F}^{n_{in}}$. If any of these messages are missed, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 6, $\mathcal{S}$ receives $n_{\times}$ AHE ciphertexts (and τ_{mul}) from $\mathcal{P}^*$. If any of these messages are missed, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 7, $\mathcal{S}$ sends a uniform $\chi \xleftarrow{\$} \mathbb{F}$ to $\mathcal{P}^*$ and computes the topology vector for each branch (depending on χ).
- In Step 8, $\mathcal{S}$ receives $n_{in} + 3n_{\times} + 1$ AHE ciphertexts (and τ_{tv}) from $\mathcal{P}^*$. If any of these messages are missed, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 9, $\mathcal{S}$ receives $n_{in} + 3n_{\times}$ AHE ciphertexts (and τ_{ip}) from $\mathcal{P}^*$. If any of these messages are missed, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 10, $\mathcal{S}$ receives $2B$ AHE ciphertexts (and τ_{mk}) from $\mathcal{P}^*$. If any of these messages are missed, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 11, $\mathcal{S}$ receives the AHE ciphertext (and τ_{pad}) from $\mathcal{P}^*$. If any of these messages are missed, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 12, $\mathcal{S}$ sends a uniform $\zeta \xleftarrow{\$} \mathbb{F}$ to $\mathcal{P}^*$. It then receives a polynomial $VP(\cdot)$ from $\mathcal{P}^*$. If $VP(\cdot)$ has degree greater than $2R-2$ or is not vanishing, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- In Step 13, $\mathcal{S}$ sends $(\text{seed}, \tau_{\text{seed}})$ and Λ to $\mathcal{P}^*$.
- For Steps 14 to 19, $\mathcal{S}$ receives $(\text{open}, \tau_{\text{wit}})$, $(\text{open}, \tau_{\text{mul}})$, $(\text{open}, \tau_{\text{tv}})$, $(\text{open}, \tau_{\text{ip}})$, $(\text{open}, \tau_{\text{mk}})$, and $(\text{open}, \tau_{\text{pad}})$ from $\mathcal{P}^*$. Since $\mathcal{S}$ previously emulated $\mathcal{F}_{\text{Com}}$, it already has the corresponding ciphertexts and can decrypt them to compute the associated IT-MACs — just as an honest verifier would in the real world.
- For Step 20, $\mathcal{S}$ computes the expected IT-MAC and receives an opening from $\mathcal{P}^*$. If the opening is invalid or the revealed value does not equal $VP(\Lambda)$, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- For Step 21, $\mathcal{S}$ performs the LPZK protocol with $\mathcal{P}^*$, if the check fails, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- For Step 22, $\mathcal{S}$ computes the expected IT-MAC and receives an opening from $\mathcal{P}^*$. If the opening is invalid or the revealed value does not equal zero, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{ZK}}^{R,B}$ and aborts.
- Finally, if $\mathcal{S}$ has not aborted, it sends the extracted witnesses $\widetilde{\mathbf{w}}_{j \in [R]}$ to $\mathcal{F}_{\text{ZK}}^{R,B}$. Since $\mathcal{F}_{\text{ZK}}^{R,B}$ also requires the active branch indices, $\mathcal{S}$ determines them by

checking which branch is satisfied by the extracted input for each repetition. If no branch satisfies some repetition, $\mathcal{S}$ sets the corresponding index to zero.

We now justify the validity of the simulator described above. As outlined in the preview, the key concern is soundness error — i.e., the case where the emulated $\mathcal{V}$ outputs `true`, but the extracted witness is invalid. To analyze this, we break it down into a sequence of potential bad events (Note that this case implies $\mathcal{P}^*$ has successfully passed all verification checks):

1. We model the PRG as an RO, introducing a negligible ϵ_{ro} error. Below, we assume the PRG output is uniform.¹³
2. Since the AHE scheme satisfies linear targeted malleability, the polynomials committed by $\mathcal{P}^*$ (using AHE) must have degree at most $2R-2$, except with negligible probability — denoted by ϵ_{ltm} — as defined by the corresponding security game. See [WYY+22] for a detailed analysis. Below, we assume that $\mathcal{P}^*$ is restricted to committing only to polynomials of degree at most $2R-2$.
3. Since Step 20 passes, according to [WYY+22, Lemma 4], all aggregated polynomials must be vanishing, except with negligible probability $\frac{2R+B+n_{\times}+2}{|\mathbb{F}|} + \epsilon_{cpa}$.¹⁴ Here, ϵ_{cpa} is defined by the CPA security game of AHE. Below, we assume that all aggregated polynomials are vanishing.
4. Since Step 21 passes, according to the soundness error of LPZK [YSWW21] (see also Appendix A.2), all multiplication IT-MAC triples must form correctly, except with negligible probability $\frac{B+n_{in}+4n_{\times}+1}{|\mathbb{F}|}$. Now, if there exists a multiplication of IT-PAC triples that are ill-formed, since the polynomials are fixed before $\mathcal{P}^*$'s learning A , this can only happen with negligible probability $\frac{2R-2}{|\mathbb{F}|}$, based on Lemma 1. Thus, all committed polynomial multiplication triples must form correctly, except with negligible probability $\frac{2R+B+n_{in}+4n_{\times}-1}{|\mathbb{F}|}$. Below, we assume that all committed polynomial multiplication triples are well-formed.
5. Since we are analyzing the scenario in which $\mathcal{P}^*$ uses an incorrect extended witness and still passes all checks, we can assume — w.l.o.g. — that the first repetition contains the invalid witness (encoded inside polynomials). We have the following facts:
 - (a) Since the polynomials corresponding to $MK_{i \in [B]}$ are vanishing, their evaluations at α_1 must produce a length- B one-hot vector with a single 1 and 0s in all other positions.

¹³ Note that in Theorem 1, we assume G is a PRG.

¹⁴ We note that the soundness error here differs slightly from that in [WYY+22]. In Antman, the random oracle (RO) is used both to accumulate IT-MAC opening proofs and to generate random linear combinations. We will revisit these differences and related variants at the end of this section. In fact, their lemma is stronger than what we require — they need to prove in ZK that multiple pairs of committed polynomials each agrees at fixed public evaluation points. One may notice that their proof does not explicitly cover linear combinations of polynomials, but this can be easily supported: since the simulator emulates the VOLE hybrid, linear combinations are handled locally without additional interaction.

- (b) Since the polynomial corresponding to committed topology vectors and $MK_{i \in [B]}$ is vanishing, this implies that the universal hashing of the first committed topology vector (i.e., evaluating all $TV(\cdot)$ s at α_1) must equal the universal hashing of the topology vector marked by the above one-hot vector. Since γ is uniformly sampled and determined after $MK(\cdot)$ s and $TV(\cdot)$ s are committed, the first committed topology vector must be some valid topology vector, except with negligible probability $\frac{n_{in}+3n_{\times}}{|\mathbb{F}|}$, based on Lemma 1. Below, we assume that the first committed topology vector is a valid topology vector from the public set.
- (c) Since the polynomials corresponding to $(L(\cdot)R(\cdot) - O(\cdot))$ s are vanishing and polynomial multiplications are well-formed, for each $k \in [n_{\times}]$, we must have $L_k(\alpha_1)R_k(\alpha_1) = O_k(\alpha_1)$.
- (d) Since the polynomial corresponding to $IP(\cdot)$ is vanishing and polynomial multiplications are well-formed, the inner product between the first extended witness and the first committed topology vector must be zero.
- (e) Now, if the first extended witness does not satisfy any branch, yet all multiplication triples are well-formed, then multiplying the topology matrices on the left by this witness must yield B non-zero length- $(2n_{\times}+1)$ vectors. However, since the inner product with the committed topology vector yields a zero, it means that there exists some non-zero vector that inner products with a uniform element and its powers yield a zero. Based on [YHH⁺23a, Lemma 1] (and its extension), this can only happen with negligible probability $\frac{2Bn_{\times}}{|\mathbb{F}|}$.

Thus, all repetitions must contain valid witnesses (encoded inside polynomials), except with negligible probability $\frac{n_{in}+(2B+3)n_{\times}}{|\mathbb{F}|}$. Below, we assume that all repetitions contain valid witnesses (encoded inside polynomials).

6. Since Step 22 passes, the committed coefficients (inside IT-MACs) must meet the ones used in the polynomial $IN(\cdot)$ s, except with negligible probability $\frac{2R-1}{|\mathbb{F}|}$ because:
 - (a) $\mathcal{P}^*$ can forge a non-zero IT-MAC to zero with probability $\frac{1}{|\mathbb{F}|}$.
 - (b) $\mathcal{P}^*$ does not know Λ when it chooses $IN(\cdot)$ s and cs, if the coefficients do not meet the polynomial, $\mathcal{P}^*$ can create a zero IT-MAC with probability $\frac{2R-2}{|\mathbb{F}|}$, based on Lemma 1.
7. Finally, note that at this point, the extracted witness of $\mathcal{S}$ must be a valid one, i.e., the ideal-world $\mathcal{V}$ would also output **true**.

To sum up, the union soundness error bound (i.e., the summed errors) is

$$\frac{6R+2B+2n_{in}+(2B+8)n_{\times}}{|\mathbb{F}|} + \epsilon_{ro} + \epsilon_{itm} + \epsilon_{cpa}$$

which is negligible. □

Cost analysis. We tally the cost of Justvengers:

- **$\mathcal{F}_{\text{com}}$ Hybrid:** $\mathcal{V}$ commits to and opens λ bits whereas $\mathcal{P}$ commits to and opens $2B+3n_{in}+10n_{\times}+2$ AHE ciphertexts.

- **$\mathcal{F}_{\text{Vole}}$ Hybrid:** Generating $Rn_{in}+2B+3n_{in}+10n_{\times}+3$ VOLE correlations.
- **Communication:** Besides hybrid calls, $\mathcal{V}$ sends $2R-2$ AHE ciphertexts and 5 $\mathbb{F}$ elements (including one implicitly used in the LPZK check) whereas $\mathcal{P}$ sends Rn_{in} $\mathbb{F}$ elements to commit input coefficients, $2R-1$ $\mathbb{F}$ elements to open the aggregated vanishing polynomial, and $n_{in}+3$ $\mathbb{F}$ elements as proofs in the verification phase.

Clearly, if $n_{in} = \mathcal{O}(1)$, the total communication is $\mathcal{O}(R+B+C)$ even in the plain model by assuming the hardness of LPN (Lemma 2 and see [WYY⁺22] for how to efficiently instantiate AHE and $\mathcal{F}_{\text{Com}}$).

- **Round:** Justvengers clearly achieves constant-round complexity.
- **$\mathcal{P}$'s Computation:** The dominant cost in $\mathcal{P}$'s computation arises from two sources: (1) $\mathcal{O}(BC)$ field operations to generate the B topology vectors, as described in Batchman, and (2) NTT-based polynomial interpolation as in Antman. Since each polynomial has degree $\mathcal{O}(R)$, interpolation requires $\mathcal{O}(R \log R)$ field operations per polynomial. With a total of $\mathcal{O}(B+C)$ polynomials to generate, the overall computational complexity for $\mathcal{P}$ is $\mathcal{O}(BC + (B+C)R \log R)$ field operations.
- **$\mathcal{V}$'s Computation:** The dominant cost in $\mathcal{V}$'s computation arises from two sources: (1) $\mathcal{O}(BC)$ field operations to generate the B topology vectors, as described in Batchman, and (2) $\mathcal{O}(R \log R)$ field operations to verify that the degree- $\mathcal{O}(R)$ polynomial $VP(\cdot)$ vanishes at R points via NTT. Thus, the overall computational complexity for $\mathcal{V}$ is $\mathcal{O}(BC + R \log R)$ field operations.

Variants. We present two variants of our protocol that may offer practical improvements, while preserving the same asymptotic complexity:

1. **Replacing Each Powers Vector with a Uniform Vector:** In our construction, $\mathcal{V}$ uses a uniform field element and its powers to define the linear combination challenge (i.e., via universal hashing). Alternatively, $\mathcal{V}$ can sample and send a fully uniform vector to perform the linear combination. While this increases communication, it decreases the soundness error. This is a common technique, as discussed in both Antman [WYY⁺22] and Batchman [YHH⁺23a].
2. **Leveraging a Random Oracle:** The linear combination can alternatively be derived by expanding a PRG seed (modeled as a random oracle). This reduces communication overhead but introduces an additional soundness error term associated with the use of the RO. This is again a common technique, as discussed in both Antman [WYY⁺22] and Batchman [YHH⁺23a].

An additional communication optimization can be achieved using a random oracle. Specifically, checking that a set of IT-MACs are zero can be reduced to computing a running hash — $\mathcal{P}$ only needs to send the final digest rather than individual zero proofs. This technique, known as the **CheckZero** procedure in Antman [WYY⁺22], is a standard optimization used in nearly all VOLE-based ZK protocols (and beyond, such as SPDZ MPC [DPSZ12]).

4.3 Soldering Repetitions

Recall that a batched disjunctive statement is uninformative if the witnesses across repetitions are uncorrelated — if $\mathcal{P}$ can prove a single repetition, simply reusing the same witness across all R instances offers no meaningful guarantee. This has concrete implications — e.g., when using the batched disjunctive statements to model a sequence of CPU steps, the instruction executed in each step depends on the results of prior computation, requiring (extended) witness consistency across repetitions.

In this section, we show how **Justvengers** can support this by showing how a concrete correlated constraint can be supported. $\mathcal{P}$ aims to prove to $\mathcal{V}$ in ZK that, for each $j \in [R] \setminus \{1\}$, the first input of the j -th repetition is equal to the first multiplication output of the $(j-1)$ -th repetition.

Casting this to polynomial encodings, this means $\mathcal{P}$ has to prove in ZK that

$$\forall j \in [R] \setminus \{1\}, IN_1(\alpha_j) = O_1(\alpha_{j-1})$$

This type of cross-repetition constraint can be handled using the “sacrifice” technique [DPSZ12]. In particular, $\mathcal{P}$ generates two random polynomials $r_1(\cdot)$ and $r_2(\cdot)$ such that the desired relation holds over them. Then, $\mathcal{P}$ reveals the one-time padded polynomials $\phi IN_1(\cdot) + r_1(\cdot)$ and $\phi O_1(\cdot) + r_2(\cdot)$ to $\mathcal{V}$, who issues a uniform $\phi \xleftarrow{\$} \mathbb{F}$ and verifies that the revealed two polynomials satisfy the intended constraint. ZK is ensured by the random padding.

We note that this “sacrifice” technique is also exploited by **Antman**, and in fact, **Justvengers** implicitly applies it in the batched vanishing polynomial check.

Acknowledgments

This work is supported in part by the Visa research award and the NSF awards CNS-2246354, and CCF-2217070. The author sincerely thanks David Heath and Chenkai Weng for their insightful discussions and valuable feedback, as well as the TCC’25 reviewers for their thoughtful and constructive comments.

References

- BBD⁺23. Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 581–615. Springer, Cham, August 2023.
- BBMH⁺21. Carsten Baum, Lennart Braun, Alexander Munch-Hansen, Benoît Razet, and Peter Scholl. Appenzeller to brie: Efficient zero-knowledge proofs for mixed-mode arithmetic and Z2k. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 192–211. ACM Press, November 2021.

- BBMHS22. Carsten Baum, Lennart Braun, Alexander Munch-Hansen, and Peter Scholl. $\text{Moz}\mathbb{Z}_{2^k}$ arella: Efficient vector-OLE and zero-knowledge proofs over $\mathbb{Z}_{2^k}$. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 329–358. Springer, Cham, August 2022.
- BCC⁺24. Dung Bui, Haotian Chu, Geoffroy Couteau, Xiao Wang, Chenkai Weng, Kang Yang, and Yu Yu. An efficient zk compiler from simd circuits to general circuits. *Journal of Cryptology*, 38(1):10, 2024.
- BCG⁺14. Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014.
- BCG⁺19. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- BCGI18. Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912. ACM Press, October 2018.
- BCI⁺22. Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. Succinct non-interactive arguments via linear interactive proofs. *Journal of Cryptology*, 35(3):15, July 2022.
- BDOZ11. Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 169–188. Springer, Berlin, Heidelberg, May 2011.
- BDSW23. Carsten Baum, Samuel Dittmer, Peter Scholl, and Xiao Wang. Sok: vector OLE-based zero-knowledge protocols. *DCC*, 91(11):3527–3561, 2023.
- Bea95. Donald Beaver. Precomputing oblivious transfer. In Don Coppersmith, editor, *CRYPTO’95*, volume 963 of *LNCS*, pages 97–109. Springer, Berlin, Heidelberg, August 1995.
- BFKL94. Avrim Blum, Merrick L. Furst, Michael J. Kearns, and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In Douglas R. Stinson, editor, *CRYPTO’93*, volume 773 of *LNCS*, pages 278–291. Springer, Berlin, Heidelberg, August 1994.
- BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012.
- BMRS21. Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 92–122, Virtual Event, August 2021. Springer, Cham.
- Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In

- Yvo Desmedt, editor, *CRYPTO'94*, volume 839 of *LNCS*, pages 174–187. Springer, Berlin, Heidelberg, August 1994.
- DEGL⁺23. Samuel Dittmer, Karim Eldefrawy, Stéphane Graham-Lengrand, Steve Lu, Rafail Ostrovsky, and Vitor Pereira. Boosting the performance of high-assurance cryptography: Parallel execution and optimizing memory access in formally-verified line-point zero-knowledge. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 2098–2112. ACM Press, November 2023.
- DILO22. Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Improving line-point zero knowledge: Two multiplications for the price of one. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 829–841. ACM Press, November 2022.
- DIO21. Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-point zero knowledge and its applications. In Stefano Tessaro, editor, *ITC 2021*, volume 199 of *LIPICs*, pages 5:1–5:24. Schloss Dagstuhl, July 2021.
- DL78. Richard A. DeMillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Inf. Process. Lett.*, 7(4):193–195, 1978.
- DPSZ12. Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multi-party computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.
- FDNZ21. Zhiyong Fang, David Darais, Joseph P. Near, and Yupeng Zhang. Zero knowledge static program analysis. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2951–2967. ACM Press, November 2021.
- GGHAK22. Aarushi Goel, Matthew Green, Mathias Hall-Andersen, and Gabriel Kaptchuk. Stacking sigmas: A framework to compose Σ -protocols for disjunctions. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 458–487. Springer, Cham, May / June 2022.
- GHAK24. Aarushi Goel, Mathias Hall-Andersen, and Gabriel Kaptchuk. Dora: A simple approach to zero-knowledge for RAM programs. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 869–883. ACM Press, October 2024.
- GHAKS23. Aarushi Goel, Mathias Hall-Andersen, Gabriel Kaptchuk, and Nicholas Spooner. Speed-stacking: Fast sublinear zero-knowledge proofs for disjunctions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 347–378. Springer, Cham, April 2023.
- GMR85. Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th ACM STOC*, pages 291–304. ACM Press, May 1985.
- HHK⁺24. Carmit Hazay, David Heath, Vladimir Kolesnikov, Muthuramakrishnan Venkitasubramaniam, and Yibin Yang. LogRobin++: Optimizing proofs of disjunctive statements in VOLE-based ZK. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 367–401. Springer, Singapore, December 2024.
- HK20. David Heath and Vladimir Kolesnikov. Stacked garbling for disjunctive zero-knowledge proofs. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 569–598. Springer, Cham, May 2020.

- HY24. Carmit Hazay and Yibin Yang. Toward malicious constant-rate 2PC via arithmetic garbling. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 401–431. Springer, Cham, May 2024.
- HYDK21. David Heath, Yibin Yang, David Devecsery, and Vladimir Kolesnikov. Zero knowledge for everything and everyone: Fast ZK processor with cached ORAM for ANSI C programs. In *2021 IEEE Symposium on Security and Privacy*, pages 1538–1556. IEEE Computer Society Press, May 2021.
- IOS23. Yuval Ishai, Rafail Ostrovsky, and Akash Shah. Succinct arguments for RAM programs via projection codes. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 159–192. Springer, Cham, August 2023.
- IOS25. Yuval Ishai, Rafail Ostrovsky, and Akash Shah. Zero-knowledge RAM: Doubly efficient and black-box. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 363–395. Springer, Cham, May 2025.
- IP07. Yuval Ishai and Anat Paskin. Evaluating branching programs on encrypted data. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 575–594. Springer, Berlin, Heidelberg, February 2007.
- JKO13. Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 955–966. ACM Press, November 2013.
- KPR18. Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 158–189. Springer, Cham, April / May 2018.
- KZG10. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010.
- LAH⁺22. Ning Luo, Timos Antonopoulos, William R. Harris, Ruzica Piskac, Eran Tromer, and Xiao Wang. Proving UNSAT in zero knowledge. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2203–2217. ACM Press, November 2022.
- LHLA25. Mengling Liu, Yang Heng, Xingye Lu, and Man Ho Au. JesseQ: Efficient Zero-Knowledge Proofs for Circuits over Any Field . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3161–3179, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- LJA⁺22. Ning Luo, Samuel Judson, Timos Antonopoulos, Ruzica Piskac, and Xiao Wang. ppSAT: Towards two-party private SAT solving. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 2983–3000. USENIX Association, August 2022.
- LKA⁺24. Daniel Luick, John C. Kolesar, Timos Antonopoulos, William R. Harris, James Parker, Ruzica Piskac, Eran Tromer, Xiao Wang, and Ning Luo. ZKSMT: A VM for proving SMT theorems in zero knowledge. In Davide Balzarotti and Wenyan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.

- LWS⁺24. Ning Luo, Chenkai Weng, Jaspal Singh, Gefei Tan, Mariana Raykova, and Ruzica Piskac. Privacy-preserving regular expression matching using TNFA. In Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas, editors, *ESORICS 2024, Part II*, volume 14983 of *LNCS*, pages 225–246. Springer, Cham, September 2024.
- LWX⁺23. Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. Zksql: Verifiable and efficient query evaluation with zero-knowledge proofs. *Proceedings of the VLDB Endowment*, 16(8):1804–1816, 2023.
- LXY24a. Fuchun Lin, Chaoping Xing, and Yizhou Yao. Interactive line-point zero-knowledge with sublinear communication and linear computation. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 337–366. Springer, Singapore, December 2024.
- LXY24b. Fuchun Lin, Chaoping Xing, and Yizhou Yao. More efficient zero-knowledge protocols over $\mathbb{Z}_{2^k}$ via Galois rings. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IX*, volume 14928 of *LNCS*, pages 424–457. Springer, Cham, August 2024.
- LXZ21. Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2968–2985. ACM Press, November 2021.
- NNOB12. Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 681–700. Springer, Berlin, Heidelberg, August 2012.
- Sch80. Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980.
- SGRR19. Phillipp Schoppmann, Adrià Gascón, Leonie Reichert, and Mariana Raykova. Distributed vector-OLE: Improved constructions and implementation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 1055–1072. ACM Press, November 2019.
- WYKW21. Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021.
- WYX⁺21. Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 501–518. USENIX Association, August 2021.
- WYY⁺22. Chenkai Weng, Kang Yang, Zhaomin Yang, Xiang Xie, and Xiao Wang. AntMan: Interactive zero-knowledge proofs with sublinear communication. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2901–2914. ACM Press, November 2022.
- Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.
- YH24. Yibin Yang and David Heath. Two shuffles make a RAM: Improved constant overhead zero knowledge RAM. In Davide Balzarotti and Wenjuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.

- YHH⁺23a. Yibin Yang, David Heath, Carmit Hazay, Vladimir Kolesnikov, and Muthuramakrishnan Venkitasubramaniam. Batchman and robin: Batched and non-batched branching for interactive ZK. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 1452–1466. ACM Press, November 2023.
- YHH⁺23b. Yibin Yang, David Heath, Carmit Hazay, Vladimir Kolesnikov, and Muthuramakrishnan Venkitasubramaniam. Batchman and robin: Batched and non-batched branching for interactive ZK. Cryptology ePrint Archive, Report 2023/1257, 2023.
- YHH⁺24. Yibin Yang, David Heath, Carmit Hazay, Vladimir Kolesnikov, and Muthuramakrishnan Venkitasubramaniam. Tight ZK CPU: Batched ZK branching with cost proportional to evaluated instruction. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3095–3109. ACM Press, October 2024.
- YHKD22. Yibin Yang, David Heath, Vladimir Kolesnikov, and David Devecsery. EZEE: Epoch parallel zero knowledge for ANSI C. In *2022 IEEE European Symposium on Security and Privacy*, pages 109–123. IEEE Computer Society Press, June 2022.
- YSWW21. Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021.
- YWL⁺20. Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated OT with small communication. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1607–1626. ACM Press, November 2020.
- Zip79. Richard Zippel. Probabilistic algorithms for sparse polynomials. In *International symposium on symbolic and algebraic manipulation*, pages 216–226. Springer, 1979.

A Deferred Preliminaries

A.1 Additively Homomorphic Encryption over $\mathbb{F}$

Definition 3 (AHE). *An additively homomorphic encryption (AHE) scheme AHE consists of algorithms (Setup, KeyGen, Enc, Dec), and is defined as follows:*

- **Setup**(1^λ): On input a (unary) security parameter λ , this algorithm outputs a set of public parameters **par**.
- **KeyGen**(**par**; r): On input the set of public parameters **par** (and a uniformly random string r), this algorithm outputs a secret key **sk**.
- **Enc**(**sk**, m ; r): On input the secret key **sk** and a message $m \in \mathbb{F}$ (and a uniformly random string r) where **par** is assumed to be an implicit input, this algorithm outputs a ciphertext c . We use $\llbracket m \rrbracket$ to denote a ciphertext over m .
- **Dec**(**sk**, c): On input the secret key **sk** and a ciphertext c where **par** is an implicit input, this algorithm outputs a message $m \in \mathbb{F}$.
- **Additively Homomorphism (over $\mathbb{F}$)**: With only **par**, one can obtain (1) $\llbracket am + b \rrbracket$ for $a, b \in \mathbb{F}$ from $\llbracket m \rrbracket$, and (2) $\llbracket m_1 + m_2 \rrbracket$ from $\llbracket m_1 \rrbracket$ and $\llbracket m_2 \rrbracket$.

In this work, we require an AHE scheme that satisfies correctness, CPA security, circuit privacy, and linear targeted malleability. For formal definitions of these properties and justification for using the standard BGV-AHE [BGV12] as a suitable instantiation, we refer the reader to [WYY⁺22].

A.2 Line-Point Zero-Knowledge (LPZK)

We review the LPZK technique [DIO21, YSWW21] used by Justvengers.

Our goal is to let $\mathcal{P}$ prove in ZK that three IT-MAC commitments form a multiplication triple. Namely, $\mathcal{P}$ and $\mathcal{V}$ hold $[x]_\Delta := \langle (x, m_x), k_x \rangle_\Delta$, $[y]_\Delta := \langle (y, m_y), k_y \rangle_\Delta$, $[z]_\Delta := \langle (z, m_z), k_z \rangle_\Delta$, where $\mathcal{P}$ wants to prove in ZK that $z = xy$. LPZK relies on the following equality:

$$\begin{aligned} & \overbrace{k_x k_y + k_z \Delta}^{\text{known by } \mathcal{V}} \\ &= (-x\Delta + m_x)(-y\Delta + m_y) + (-z\Delta + m_z)\Delta \\ &= \underbrace{(xy - z)}_0 \Delta^2 + \underbrace{(-xm_y - ym_x + m_z)}_{\text{known by } \mathcal{P}} \Delta + \underbrace{m_x m_y}_{\text{known by } \mathcal{P}} \end{aligned}$$

Hence, if ZK is not required, $\mathcal{P}$ can send two coefficients $C_1 = -xm_y - ym_x + m_z$ and $C_0 = m_x m_y$, and $\mathcal{V}$ will check if $k_x k_y + k_z \Delta \stackrel{?}{=} C_1 \Delta + C_0$. This is sound because the equality holds only when Δ happens to be the root of a quadratic equation if $xy \neq z$. If Δ has full entropy to $\mathcal{P}$, this will only happen with probability $\frac{2}{p}$ based on Lemma 1. Of course, this is not ZK as C_1, C_0 are correlated with x, y, z . ZK can be recovered by consuming one random IT-MAC $[r]_\Delta$ (from VOLE). That is, $\mathcal{P}$ sends $C_1 = -xm_y - ym_x + m_z - r$ and $C_0 = m_x m_y + m_r$ and $\mathcal{V}$ checks if $k_x k_y + k_z \Delta + k_r \stackrel{?}{=} C_1 \Delta + C_0$.

The LPZK technique can be optimized for batch settings. Specifically, suppose the parties hold $[x]_\Delta$, $[y]_\Delta$, and $[z]_\Delta$, each of length n , and the goal is for $\mathcal{P}$ to prove in ZK that $z = x \odot y$. Instead of sending $2n$ individual coefficients, the n checks can be batched as follows: $\mathcal{V}$ sends a random linear combination, and $\mathcal{P}$ evaluates this combination over the n coefficients C_1 s and C_0 s, and then sends resulting 2 aggregated coefficients. As before, a single random IT-MAC $[r]_\Delta$ is required for ZK. The random linear combination can be replaced by powers of a random element.

A.3 Batchman [YHH⁺23a]

Topology matrices and vectors. Consider a circuit $\mathcal{C}$ with n_{in} inputs and $n_\times$ multiplication gates. Note that a ZKP for $\mathcal{C}$ can be separated into two parts: (1) multiplication gates and (2) linear constraints. Suppose that $\mathcal{P}$ commits — using IT-MACs — to its input $[in_1], \dots, [in_{n_{in}}]$, and also commits to the values on the $3n_\times$ wires associated with $\mathcal{C}$'s $n_\times$ multiplication gates. That is, $\mathcal{P}$ commits to $[\ell_1], \dots, [\ell_{n_\times}]$, corresponding to the multiplication left input wires, to

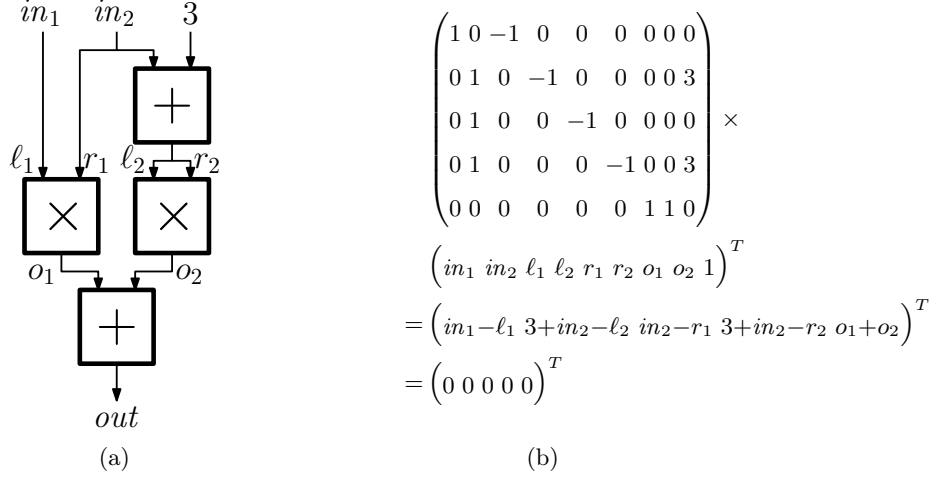


Fig. 10: (a) A circuit of $(in_1 \cdot in_2) + (in_2 + 3)^2$ and (b) its topology matrix.

$[r_1], \dots, [r_{n_\times}]$, corresponding to the right input wires, and to $[o_1], \dots, [o_{n_\times}]$, corresponding to the output wires. The full vector of $\mathcal{P}$'s input and the multiplication wires (with a constant 1) is called $\mathcal{P}$'s extended witness.

Now, $\mathcal{P}$ first proves to $\mathcal{V}$ that $\ell \odot r = o$, demonstrating that its extended witness satisfies multiplicative constraints. Then, it proves that $\mathbf{in}, \ell, r, o$ indeed respect the linear constraints imposed by circuit $\mathcal{C}$. Note that since all multiplication gates were handled in the first step, $\mathcal{P}$ simply needs to show its extended witness respects a particular linear relation — i.e. a matrix M . This public matrix M is induced by the structure of the circuit $\mathcal{C}$, and [YHH⁺23a] refers to M as a *topology matrix*. Namely, $\mathcal{P}$ proves the following:

$$M \times (\mathbf{in}, \ell, r, o, 1)^T = \mathbf{0} \quad (1)$$

Since $\mathbf{in}, \ell, r, o$ are committed, this equality check can be handled by $\mathcal{V}$'s sending of a uniform challenge $\chi \xleftarrow{\$} \mathbb{F}$ where parties locally construct the IT-MAC of

$$\underbrace{(1, \chi, \dots, \chi^{2n_\times})}_{\text{topology vector}} \times M \times \underbrace{(\mathbf{in}, \ell, r, o, 1)^T}_{\text{extended witness}} \quad (2)$$

and then proves to $\mathcal{V}$ that this is a commitment to 0. Recall that M is public, so once χ is fixed, both $\mathcal{P}$ and $\mathcal{V}$ know $(1, \dots, \chi^{2n_\times}) \times M$ (called a *topology vector*). Thus, it suffices to check whether the inner product between the topology vector and the extended witness yields 0. Figure 10b shows an example topology matrix.

Proving batched disjunctions: Batchman [YHH⁺23a]. The above paradigm is an overkill if we only perform a ZKP for a single circuit. This is because it is worse than the state-of-the-art VOLE-based CPZK (e.g. QuickSilver [YSWW21]), which only requires committing $\mathbf{in}$ and $\mathbf{o}$. However, this paradigm becomes useful when considering a batch of disjunctions, as observed by Batchman [YHH⁺23a].

In detail, Batchman [YHH⁺23a] considers B different circuits $\mathcal{C}_1, \dots, \mathcal{C}_B$ of the same size. $\mathcal{P}$ wants to repeat the disjunctive proof R times — for each $j \in [R]$, she proves that she knows some witness $\mathbf{w}_j$ and some index $id_j \in [B]$ such that $\mathcal{C}_{id_j}(\mathbf{w}_j) = 0$. To achieve this, for the j -th repetition, $\mathcal{P}$ commits to her extended witness of *only* $\mathcal{C}_{id_j}$. $\mathcal{V}$ then issues a uniform challenge χ to compress B topology matrices to B topology vectors. The *crucial step* is that, for the j -th repetition, $\mathcal{P}$ can commit to the id_j -th topology vector. An extra mechanism is needed to prevent $\mathcal{P}$ from committing to an arbitrary vector that is not a topology vector, which can be built based on a ZK ROM (e.g., [YH24]). Finally, it suffices to show that the inner product between the extended witness and the topology vector is 0 for each repetition (e.g., by using the generalized LPZK [YSWW21]).